QF600 - Asset Pricing Project Report:

Extension of the Black-Litterman Model with Multi-Benchmark Tracking Error

BY

Angelica Gloria Bong
Aryan Goel
Cheery Aldora Suvaco
Grace Chandra
Karan Rajesh Chavan
Sim Jaehyun

UNDER THE GUIDANCE OF
Dr. Peng Liu

MSc in Quantitative Finance

SINGAPORE MANAGEMENT UNIVERSITY

November 2025

Executive Summary

For the longest time, traditional portfolio optimization frameworks have often relied on a single benchmark, resulting in overfitting, benchmark dependency, and limited diversification. These limitations hinder the practical applications of these models in dynamic markets, where investors are evaluated against multiple reference indices (Markowitz, 1952; Chen et al., 2021). To address these challenges, this research integrates the Black-Litterman (BL) model with the Multi-Benchmark Tracking Error (MBTE) approach, forming a unified framework, MBTE-BL, that merges Bayesian inference with adaptive benchmark tracking. This paper aims to address the limitations of each model, reconciling BL's structured subjectivity with MBTE's empirical adaptiveness, thereby producing a more stable approach that yields, weights, and superior risk-adjusted returns through the integration of the BL-MBTE Model. The data collected is 50 stocks selected from the S&P 500 Index with 5 sector factors. Additionally, 9 ETFs are chosen to be the performance benchmark. All data were sourced from yfinance. The final result shows that the combined approach outperformed the Black-Litterman model; however, it still requires refinement and tuning, as well as research on its low performance compared to the MBTE model. It is possibly due to the limitations, such as allocation sensitivity and equal-weight treatment for the benchmark. Hence, it is possible to consider, for example, replacing prior with free-float market cap and considering different weights for each benchmarks (Thompson Sampling) in the future works of this study.

1. Introduction

Historically, portfolio optimization and evaluation were designed relative to a single market representation, typically a capitalization-weighted index, such as the S&P 500. Since Markowitz's work in 1952, which formalized the mean–variance optimization framework, modern finance has sought to reconcile theoretical elegance with empirical robustness. Although Markowitz's framework revolutionized the understanding of diversification and risk-return trade-offs, its practical implementation revealed severe estimation fragility: small variations in expected return inputs can lead to disproportionately different portfolio weights, often yielding unstable or counterintuitive outcomes. Nowadays, modern institutional asset management operates within complex investment universes where performance must align with multiple benchmarks reflecting diverse investment mandates, styles, and exposures. The dominance of one benchmark routinely shifted across economic cycles due to changes in factors such as monetary policy, sectoral transitions, and technological innovation. Consequently, the single-benchmark paradigm imposes rigidity, leading to systematic biases and inconsistent performance when market leadership transitions occur. The need to reconcile performance evaluation and optimization across multiple reference indices has therefore become integral to modern portfolio design.

Two major frameworks have emerged to address these challenges: the Black–Litterman (BL) model and the Multi-Benchmark Tracking Error (MBTE) framework. The BL model introduced a Bayesian structure to portfolio optimization, integrating market-derived equilibrium returns with subjective investor views in a coherent probabilistic framework. However, the model remains bounded by its foundational assumption of a single equilibrium benchmark portfolio. In contrast, the MBTE model replaces the single-benchmark constraint with a probabilistic ensemble of competing benchmarks, optimizing allocations to minimize deviations from the best-performing benchmark while ensuring that no individual index dominates the others. Despite its empirical success, MBTE is inherently reactive, relying solely on historical persistence probabilities without a mechanism to incorporate forward-looking market insights or investor convictions.

Recognizing these complementary strengths and limitations, this research introduces an integrated framework, the MBTE–BL model, combining the Bayesian forecasting structure of Black–Litterman with the adaptive, regime-resilient design of MBTE. The framework embeds factor-enhanced posterior returns from the BL model within MBTE's optimization process, using MBTE's probabilistic weights to refine the confidence structure of the Bayesian prior. This combination produces portfolios that are simultaneously forward-looking and dynamically adaptive, reducing estimation risk without sacrificing flexibility. Empirical evaluations using fifty S&P 500 constituents, nine factor exposures, and five distinct benchmarks (S&P 500, Equal Weight S&P 500, Value, Growth, and NASDAQ 100) demonstrate that the integrated model outperforms standalone BL, but is still lacking compared to MBTE.

2. Literature Review

Each investor has different and unique needs. Thus, it is essential to construct a suitable portfolio that accommodates their objective, as a good portfolio is more than merely a list of stocks and bonds (Markowitz, 1952). Thorough portfolio optimization is crucial when constructing a proper portfolio. Two key parameters, expected returns and risk, play a major role in optimizing a portfolio. These two parameters account for the distribution of wealth across assets and set a tradeoff between sacrificing maximum return for the lowest possible risks (Chen et al., 2021). Nevertheless, with the rise of performance evaluation relative to multiple benchmarks and the attempt to tackle systematic risk, a traditional approach to single-benchmark optimization is becoming increasingly insufficient for optimizing a portfolio. It led to the emergence of multi-benchmark tracking error and a generalized version of the Black-Litterman model.

This section reviews four existing papers that extend the classical mean-variance optimization approach: “Tracking the Best: A Multi-Benchmark Approach to Portfolio Construction”, “Tracking-Error Models for Multiple Benchmarks: Theory and Empirical Performance”, “A Generalized Black–Litterman Model”, and “Risk Factor, Risk Premium and the Black–Litterman Model”.

2.1. Evolution from traditional to multi-benchmark tracking error models

The traditional approach to portfolio optimization involves the use of a single benchmark, reflecting the raw return at a given level of risk, to minimise tracking error. However, it is not practical or sufficient because this indicates that investors are only focusing on absolute performance, whereas in practice, the focus is on surpassing the market average (Blitz et al., 2020).

Thus, as proposed by Xu et al. (2014) and Liu et al. (2025), a robust framework is established to minimize estimation errors and agency issues caused by a single benchmark tracking error, known as the Multi-Benchmark Tracking Error (MBTE) model. According to Xu et al. (2014), the MBTE portfolio exhibits less volatility, as the persistency values mitigate estimation errors, resulting in fewer anomalies compared to the single-index benchmark.

Additionally, MBTE can mitigate overtrading that occurs when maximizing alpha. As MBTE is a more alpha reserve model, it lowers transaction cost as MBTE can prevent unnecessary trading and control transaction volume (Xu et al., 2014). To support the findings from Xu et al., a recent study by Liu et al. on MBTE addresses overfitting, using a single index and reducing bias. The addition of Thompson sampling for portfolio allocation also complements the idea of mitigating the risk of increasing opportunity cost and estimation errors by choosing a non-optimal strategy, as it ensures that the portfolio continues to meet the expected return (Liu et al., 2025). This mechanism provides a clear economic intuition by balancing the

estimation risk inherent in sophisticated models against the benefits of dynamic strategy selection. It is a more empirical approach compared with the traditional intuitive methods in portfolio allocation, as Liu et al. (2025) explain that the portfolio's weight is constructed based on the probability of dominance from each benchmark. As a result, it tracks the best-performing benchmark and minimizes deviation.

2.2. Generalized Black-Litterman Model

Moving to the Black-Litterman model, the classical Black-Litterman model is an enhancement of Markowitz's mean-variance model, introducing a Bayesian framework for forecasting the expected return using the CAPM model and incorporating the investor's views on how the market will behave (Chen & Lim, 2020). The main concern with the classical model is that it assumes the prior or market-implied returns and the investor's subjective views are unbiased, where the risk is already embedded in the prior. Another possible issue that may arise is the accuracy of confidence levels for the market behaviour, as the entire model relies heavily on estimations. As argued by Huang et al. (2023), investors typically lack sufficient knowledge of stock returns or macroeconomic variables, and the framework of deterministic volatility models is highly restrictive in financial time series. Therefore, many have proposed an updated version of the classical model, known as the generalized Black-Litterman model, one of which is Chen and Lim's work.

In their paper, the generalized version addresses the shortcomings of the classical model, including model misspecification and the omission of unmodeled factors (such as risk and bias). Chen and Lim (2025) further noted that to obtain a proper forecast of the return, the history of returns and views needs to be utilized as an estimator for bias. By acknowledging that the views may be biased, they can model where different views correspond randomly throughout the period. Moreover, the equilibrium return is forecasted by acknowledging that the expected return is dynamic and moves accordingly with independent samples of a normal random variable (Chen and Lim, 2020).

2.3. Identifying Gaps

From the previous sections, the definition and background of the Black-Litterman model and MBTE have been provided. It is noticeable that the Black-Litterman model has an issue concerning bias and subjectivity. To overcome systematic risks, there has been a growing emphasis on incorporating these risk factors into contemporary portfolio optimization. By incorporating risk factors such as GDP, inflation, market value, and others, it is possible to construct a stable and economically grounded portfolio (Rjaily et al., 2024). Supporting the ideas from the Generalized Black-Litterman model, it combines a multi-factor model to address the issue of having views on a large number of assets. Rjaily et al. (2024) further noted that exposure to underlying risk factors becomes a driver of asset returns. Additionally,

incorporating a multi-factor model into the Black-Litterman model helps simplify the complex dimensionality of asset return variations.

By analyzing these papers, we propose an alternative model that is potentially applicable by combining the multi-factor models of the Generalized Black-Litterman model with the multi-benchmark tracking error model. We aim to achieve the best of both worlds and strike a balance in our approach. Hence, we will get a model that is alpha-seeking but not in an aggressive manner, where overtrading may occur. Moreover, the portfolio is constructed to be more effective, as the combined model is expected to exhibit better performance than either the single benchmark or the single-factor model. Hence, we attempt to justify this hypothesis.

3. Methodology

In this section, we will discuss our methodology, including data selection, preparation, and modelling approaches we used in our project. The methodology is designed to ensure reproducibility and a balanced evaluation of the proposed Black-Litterman and Multi-Benchmark Tracking Error (MBTE) frameworks, both individually and in combination.

3.1. Data Selection and Preparation

3.1.1. **Stock Selection**

The asset in this project comprises 50 stocks selected from the S&P 500 Index. The S&P 500 was chosen as the sampling framework due to several key attributes:

- Superior Market Liquidity facilitates efficient trade execution
- Broad Market representativeness as a benchmark for U.S. large-cap equities
- High-quality data availability ensures robust empirical analysis

By constraining the universe to large-cap, institutionally followed firms, this project ensures that the portfolio construction methodology yields insights that are both theoretically sound and practically implementable within a real-world investment framework. This approach mitigates potential concerns regarding liquidity constraints, transaction costs, and market impact that might arise from including smaller or less liquid securities.

3.1.2. **Factor Selection**

To systematically capture the principal sources of equity return variation, we employ a multi-factor framework that incorporates five sector factors: Technology, Financials, Energy, Industrials, and Healthcare, with four style factors: Value, Growth, Momentum, and Size.

Sector factors enable the identification and segregation of industry-specific drivers of portfolio risk and return, providing granular insight into the portfolio's sensitivity to various economic regimes and business cycle dynamics. This sectoral decomposition facilitates a comprehensive understanding of how macroeconomic conditions propagate through different industries to influence portfolio performance.

Style factors exhibit a complementary analytical function by distinguishing between returns attributable to systematic style exposures and those arising from pure security selection skill, which is referred to as "Alpha". This decomposition is essential for performance attribution, as it prevents the misattribution of factor-driven returns to active management skills, thereby enabling a more accurate assessment of portfolio manager effectiveness.

By combining all key features of both sector and style factors, this 9-factor structure provides a comprehensive framework for understanding the fundamental risk and return characteristics of the constructed portfolios.

3.1.3. Benchmark Selection

Nine Exchange-Traded Funds (ETFs) were designated as performance benchmarks, encompassing both equity and alternative asset classes:

Equity Benchmarks:

- SPY (SPDR S&P 500 ETF Trust) – Market-capitalization-weighted broad market exposure
- RSP (Invesco S&P 500 Equal Weight ETF) – Equal-weighted S&P 500 alternative
- IVE (iShares S&P 500 Value ETF) – Value style orientation
- IVW (iShares S&P 500 Growth ETF) – Growth style orientation
- QQQ (Invesco QQQ Trust) – Technology-concentrated Nasdaq-100 exposure

Alternative Asset Benchmarks:

- TLT (iShares 20+ Year Treasury Bond ETF) – Long-duration sovereign debt
- GLD (SPDR Gold Trust) – Commodity exposure via gold bullion
- HYG (iShares iBoxx High Yield Corporate Bond ETF) – Below-IG credit
- LQD (iShares iBoxx Investment Grade Corporate Bond ETF) – IG credit

This comprehensive multi-asset benchmark framework was constructed to serve several analytical purposes.

First, it reduces selection bias by evaluating strategy performance against a diverse set of investable alternatives rather than a single, potentially favorable comparison point.

Second, the inclusion of fixed income and commodity benchmarks enables assessment of risk-adjusted performance relative to traditional portfolio diversifiers, providing insight into whether the equity strategy delivers sufficient compensation for its inherent equity risk premium.

Third, this specification reflects institutional investment practice, where portfolio managers are routinely evaluated against multiple benchmarks spanning asset classes to assess both absolute and relative value propositions.

The equity benchmarks collectively span key style dimensions (value/growth, concentration/diversification, equal/market-cap weighting). In contrast, the alternative asset benchmarks represent the primary non-equity allocations in multi-asset portfolios. This structure facilitates a comprehensive evaluation of strategy effectiveness across varying market conditions and investor risk preferences.

3.1.4. Data Collection and Processing

All data were sourced using the Yahoo Finance API (YFinance) Python library from January 2015 to January 2025. This ten-year window provides multiple market regimes, including bull markets, bear markets, and periods of elevated volatility, thereby providing a robust empirical foundation for out-of-sample performance evaluation.

Data Collection & Preprocessing process included:

- Daily open, high, low, close (OHLC), and adjusted close prices for all stocks and benchmark ETFs
- Automated exception handling to address API connectivity failures and data transmission errors
- Data quality screening required a minimum 90% completeness threshold; securities failing this criterion were excluded to preserve the integrity of the time series.

Moreover, corporate action adjustments were implemented to ensure data accuracy and integrity. The '*auto_adjust = False*' setting was employed to retain unadjusted OHLC, with manual adjustment protocols applied for material corporate events, including stock splits and ticker symbol changes (e.g., Facebook: FB ~ Meta: META). This approach preserves price level information critical for certain technical indicators while maintaining data consistency across the full period.

Missing observation from non-trading days, market closures, or data provider (Yfinance API) gaps were addressed through forward-fill ('*.ffill*') interpolation, ensuring complete and aligned time series for subsequent portfolio construction and performance analysis.

3.1.5. Data Analysis

Correlation analysis was performed across three dimensions, which are individual stocks, risk factors, and benchmarks, revealing the diversification structure inherent in the dataset.

Stock-level analysis of 50 constituents reveals an average pairwise correlation of 0.378 (σ = 0.117, range: 0.068–0.894), indicating moderate co-movement and potential for meaningful diversification through individual security selection.

Factor-level analysis shows higher interdependence, with an average correlation of 0.705 (σ = 0.139, range: 0.439–0.922). It reflects the systematic nature of sector and style factors, which respond to common macroeconomic drivers, limiting diversification benefits within equity-only factor frameworks.

Benchmark-level analysis demonstrates average correlation of 0.422 (σ = 0.408, range: -0.259–0.978). The presence of negative correlations stems from the inclusion of alternative assets (TLT, GLD), confirming their traditional role as equity diversifiers. Near-unity correlations among equity benchmarks reflect shared systematic risk exposures.

These patterns shown in Appendix 1 confirm that diversification benefits primarily arise from security selection and cross-asset allocation, rather than factor rotation within equities.

A three-state Hidden Markov Model (HMM) was applied to SPY returns to classify the 2015-01-01 to 2025-01-01 into Bull, Bear, and Mixed market regimes based on return and volatility patterns.

Regime Structure: The sample period exhibits a balanced distribution:

Bull markets (42.4%, ~820 days), Mixed markets (42.4%, ~820 days), and Bear markets (15.3%, ~300 days).

This distribution reflects the predominantly expansionary decade, punctuated by brief but severe drawdowns, notably the 2018 Q4 correction, the March 2020 COVID-19 dislocation, and the 2022 rate-hiking cycle.

Temporal Dynamics: The regime evolution captures economically meaningful market phases:

Stable Bull conditions (2015–2017), Heightened volatility and regime transitions (2018–2020), Rapid recovery (2020–2021), Extended stress period (2022), and Renewed expansion (2023–2025). This classification aligns with major macroeconomic and policy events.

Validation: VIX levels show clear regime dependence, confirming the model's ability to identify distinct risk environments. Moreover, SPY-TLT correlations show regime-contingent behavior consistent with flight-to-quality dynamics during market stress.

This framework, as shown in Appendix 2, enables conditional performance analysis across diverse market conditions, with sufficient observations in each regime to support robust statistical inference.

3.2. Modelling

The modelling stage involves replicating established models (Black-Litterman and MBTE) and developing an integrated framework that combines both approaches. Model performance will be compared across three configurations: the Black-Litterman model, the MBTE model, and our proposed combined model.

3.2.1. **Replication**

To assess our model’s performance, we are comparing it with the performance of the Black-Litterman model and the MBTE model on their own.

3.2.1.1. Black-Litterman Model

The Black-Litterman model integrates equilibrium market information with subjective investor views using a Bayesian framework.

- Prior belief (π): Derived from market-cap weights under the CAPM equilibrium assumption, representing the market's implied expected returns.
- New evidence: Incorporates investor views (expectations about asset performance) expressed as linear combinations of asset returns, with corresponding confidence levels.
- Posterior belief (μ_{BL}): Updated expected returns that blend market equilibrium with the views, weighted by their relative certainty.

$$\mathbb{E}[R] = \mu_{BL} = \left[(\tau\Sigma)^{-1} + P'\Omega^{-1}P\right]^{-1}\left[(\tau\Sigma)^{-1}\pi + P'\Omega^{-1}Q\right]$$

In our replication, we implement the classical Black-Litterman model, following the original Bayesian formulation by Black and Litterman (1992). The model combines equilibrium returns implied by the market portfolio with subjective investor views, moderated by a confidence parameter.

In our implementation, the equilibrium (market) portfolio is defined through the market_caps argument in the optimizer. When capitalization data are unavailable, the model defaults to an equally weighted market portfolio. In cases where actual market capitalization data are provided, these values are normalized to sum to one, thereby reflecting the true market-cap-weighted equilibrium. The equilibrium returns derived from these weights serve as the prior in the Black–Litterman Bayesian updating process.

The equilibrium returns are computed as $\pi = \delta\Sigma w_{mkt}$, where δ represents the degree of risk aversion. The uncertainty scaling factor τ is used to adjust the confidence in the equilibrium prior. Investor views are encoded through a matrix P and a view vector Q, with associated uncertainty Ω. The posterior expected returns are then obtained via Bayesian updating as shown in the equation above. These posterior returns reflect both market consensus and the investor's subjective opinions. They serve as the input for the expected return in subsequent optimization routines, where we derive portfolio weights under various risk–return objectives.

3.2.1.1.1. View Construction

The model begins by identifying two representative, highly liquid assets from the available dataset, typically AAPL, MSFT, JPM, or XOM. These assets are chosen because they are large-cap securities that often serve as market leaders. If fewer than two of these tickers are present, the first two assets in the dataset are selected by default.

Once identified, a 2×N view matrix P is created, where N is the total number of assets. Each row in P represents a separate investor view, indicating which asset(s) the view pertains to. In our case, both views concern individual assets rather than relative spreads or combinations.

The expected excess returns associated with each view are captured in the vector Q. In our replications, we set Q to be $[0.02 \ -0.01]^T$. It reflects the belief that the first asset is expected to outperform the market by 2%, a mild bullish statement. In comparison, the second asset is expected to underperform by 1%, a slightly bearish view. These values are hardcoded and were chosen to be conservative, avoiding extreme deviations from market equilibrium while still generating meaningful adjustments in the posterior estimates. They also ensure that the example remains generalizable, rather than being dependent on any specific macroeconomic forecast.

The degree of confidence in each view is encoded in the diagonal covariance matrix Ω, set to diagonal(0.001, 0.001). To indicate high confidence, we use small values (0.001), which translates to giving more weight to the investor's beliefs compared to the prior (market equilibrium). For simplicity, we are implementing independent views only, which is reflected in the absence of off-diagonal elements.

3.2.1.1.2. Bayesian Updating

Inputs (P, Q, and Ω) are passed into the method update_views(P, Q, omega), which adjusts the equilibrium (market-implied) expected returns and covariance matrix to reflect the new information. The resulting posterior returns tilt the optimized portfolio toward the first asset (overweight) and away from the second (underweight), a quantitative expression of the investor's subjective beliefs.

3.2.1.1.3. Parameter Selection

Empirically, the common range used for risk aversion (δ) is between 2 and 4. In our case, we set risk aversion to three (3.0), a reasonable middle ground reflecting a moderately risk-averse market investor.

The commonly suggested uncertainty scaling factor (τ) is between 0.01 and 0.05. We chose this factor to be 0.05 to indicate a moderate level of uncertainty in the equilibrium returns. 0.05 is small enough to maintain stability, but adequate to allow meaningful updates from views.

3.2.1.2. MBTE Model

The Multi-Benchmark Tracking Error (MBTE) model extends the traditional single-benchmark tracking error framework by introducing multiple benchmarks to represent diverse market exposures. Instead of minimizing deviation from a single reference index, MBTE dynamically adjusts portfolio weights to track a composite benchmark portfolio formed from multiple benchmarks, each capturing different aspects of market performance.

Traditional tracking error models aim to minimize the deviation between the portfolio's return and that of a single benchmark return. However, in modern markets, different benchmarks perform better under different conditions (ex., SPY in broad bull markets, QQQ in tech-led rallies).

3.2.1.2.1. Initialization and Benchmark Integration

Our code begins by converting benchmark returns into a Pandas DataFrame structure to handle multiple benchmarks. The annualized benchmark returns and covariance matrix are computed. This setup enables the MBTE optimizer to anchor its optimization process based on selected benchmarks, constraining or evaluating portfolio behaviour in relation to the chosen benchmarks.

3.2.1.2.2. Tracking Error Evaluation and Parameter Constraints

Tracking error (TE) measures how much a portfolio's returns deviate from its benchmark. A low TE indicates the portfolio closely follows the benchmark's return pattern, while a high TE indicates deviation. For multiple benchmarks provided, the code calculates the average annualized tracking error across them, ensuring a balanced comparison across several market references.

For multiple benchmarks, our function computes the TE for each benchmark and takes the average TE as the optimization target. The optimizer (scipy.optimize.minimize) adjusts weights to minimize the average TE. Constraints are set so that weights sum to 1, and no short-selling is implemented (each weight between 0 and 1). These constraints ensure that the optimization yields feasible portfolios that align with realistic investment mandates. The equal-weighted starting point for optimization eliminates bias toward any benchmark component, thereby facilitating faster convergence. Results remain interpretable and directly comparable to those from the Black-Litterman and combined frameworks.

3.2.2. Combined Model Creation

The Combined Black–Litterman + Multi-Benchmark Tracking Error (BL–MBTE) framework integrates informed expected returns from the Black–Litterman model with benchmark discipline from the MBTE model. The goal is to construct a portfolio that strikes a balance between generating alpha (tilt) and conforming to the market (tether).

3.2.2.1. Conceptual Integration

The combined model begins by taking the posterior expected returns and adjusted covariance matrix from the Black-Litterman process. Instead of optimizing directly, the combined framework introduces an additional objective of minimizing multi-benchmark tracking error, effectively constraining the expressions of active views to remain benchmark-aware. In essence, the BL component provides an informational advantage, while the MBTE structure serves as a risk anchor, preventing excessive divergence from market reality.

3.2.2.2. Implementation Structure and Parameter Selection

In the code, the combined optimizer initializes both BL and MBTE objects, sequentially feeding the outputs of the BL module into the MBTE optimization routine. Specifically, the BL posterior mean (bl_returns) replaces the traditional mean return vector in the tracking error minimization problem.

The optimization itself employs scipy.optimize.minimize, targeting the minimization of the average annualized tracking error (computed using the helper function tracking_error(weights, asset_returns_df, benchmark_returns)), subject to the same portfolio constraints as in the MBTE-only model (weights sum to 1 and no short-selling). This structure ensures comparability between models.

A central feature of our combined model is the introduction of a blending parameter $\alpha \in [0,1]$, which governs the degree of integration between the BL and MBTE components. The parameter functions as a tilt–tether control mechanism. For $\alpha = 1$, the optimizer fully adheres to the BL model. Conversely, at $\alpha = 0$, the optimizer reduces to a pure MBTE formulation. Intermediate values of α create a convex combination between these two extremes, allowing controlled incorporation of subjective beliefs within a benchmark-aware structure.

In our implementation, we set $\alpha = 0.6$ for the mean–variance and Sharpe-based optimizations, reflecting a moderate level of active tilt consistent with conventional institutional mandates. For the higher-moment optimization, α was increased slightly to 0.65, allowing for greater flexibility in capturing skewness and kurtosis effects, which often require more active deviation to exploit asymmetric or non-linear risk premia.

To maintain consistency and ensure comparability, key parameters from both models are preserved. Namely, parameters in the Black-Litterman model of risk-aversion (δ), uncertainty scaling factor (τ) and covariance matrix (Ω) are set to 3.0, 0.05 and diagonal(0.001 0.001), respectively.

3.3. Unified BL–MBTE Framework

3.3.1. Integration within the Unified BL–MBTE Model

Black–Litterman posterior:

$$E[R] = [(\tau\Sigma)^{-1} + P^T\Omega^{-1}P]^{-1}[(\tau\Sigma)^{-1}\Pi + P^T\Omega^{-1}Q]$$

MBTE objective:

$$\min_x E[(r^T x - Z^B(r))^2]$$

Unified BL–MBTE solution:

$$x^* = p^B + \frac{E[Z^B] - E[R]^T p^B}{A'C' - (B')^2}\Sigma^{-1}(C'E[R] - B'e)$$

3.3.2. Empirical Observation

Ten-year S&P data (2015–2025) (Appendix 3) shows that BL–MBTE yields higher average Sharpe ratios, mildly positive skewness, and lower excess kurtosis than either component model alone.

4. Results and Discussion

4.1. Return and Risk Metric Computation

All asset-level statistics were computed from adjusted closing prices obtained through *yfinance*.

4.1.1. Return Computation

Daily returns were calculated as the percentage change in adjusted closing prices. Missing values from market holidays or data gaps were dropped before further processing. Mean daily returns were multiplied by 252 (the approximate number of trading days per year) to obtain annualized expected returns, providing a directly comparable measure across assets and benchmarks.

4.1.2. Covariance and Correlation Matrices

The covariance matrix of daily returns was scaled by 252 to yield annualized covariances, while the correlation matrix was computed in its raw daily form. These matrices quantify how assets move together and serve as inputs to both risk estimation and portfolio optimization.

4.1.3. Higher-Moment Statistics

Skewness and excess kurtosis were computed from daily returns to capture non-normal features of asset distributions. Skewness reflects asymmetry in return distributions, while excess kurtosis measures the degree of tail risk (fat-tailedness). These metrics were later incorporated into the higher-moment optimizer to penalize negative skewness and extreme kurtosis.

4.1.4. Risk-Free Rate Usage

The 13-week U.S. Treasury bill (^IRX) was used to approximate the current short-term risk-free rate, with annualization and a 60-day rolling average applied to smooth noise. The resulting annualized yield over the sample period averages approximately 3.96%.

For consistency across all model configurations, a fixed risk-free rate of 2% was used in portfolio optimization and performance evaluation. This standardization ensures that Sharpe ratios and other risk-adjusted metrics remain comparable across different model architectures and time windows, avoiding distortions caused by regime-dependent fluctuations in Treasury yields. The 3.96% value serves an informational rather than computational role in this analysis.

4.2. Portfolio Performance Analysis

Portfolio performance was evaluated across three distinct optimization frameworks: Mean–Variance (MV), Maximum Sharpe Ratio, and Higher Moments, for our three configurations: BL-only, MBTE-only, and the combined BL–MBTE model. The results are presented in terms of annualized expected return, volatility, Sharpe ratio, and tracking error.

4.2.1. Mean-Variance Portfolios

Under the mean–variance setting (Appendix 4), the MBTE-only portfolio achieves the highest Sharpe ratio (0.72), indicating superior risk-adjusted performance relative to both the BL-only and combined configurations. Consistent with MBTE's benchmark-aligned nature, its optimization objective explicitly minimizes deviation from diversified reference indices, resulting in more stable and market-like returns.

By contrast, the BL-only portfolio exhibits a low expected return and a negative Sharpe ratio, reflecting the limited scope of the two hard-coded investor views and their associated confidence levels. The BL model, when used in isolation, yields an overly conservative allocation, as equilibrium priors dominate the posterior estimates due to the small number of views.

The combined BL–MBTE portfolio outperforms the BL-only case, demonstrating that incorporating benchmark awareness stabilizes performance (Sharpe ratio = 0.21 vs. –0.91) while still allowing for modest deviations from equilibrium. It supports the intended hybrid function of the combined model—enhancing benchmark alignment while preserving view-based adjustments.

4.2.2. Maximum Sharpe Ratio Portfolios

A higher Sharpe implies superior risk-adjusted performance. In the Sharpe-ratio maximization context (Appendix 5), all models deliver positive risk-adjusted returns, with the MBTE-only model again showing the strongest performance (Sharpe ratio = 0.69). Its relatively low tracking error (12.23%) indicates tight alignment with benchmarks, consistent with a passive-plus style.

The BL–MBTE portfolio performs slightly below MBTE-only but with noticeably lower volatility (4.14% vs. 17.09%), implying smoother return dynamics. This suggests that integrating BL-derived expected returns tempers risk-taking while maintaining competitive Sharpe efficiency.

Although the BL-only configuration produces similar Sharpe levels, it remains more volatile in terms of tracking error (19.81%), as it lacks explicit benchmark tethering. The combined framework thus achieves a balanced compromise: slightly lower return potential but higher stability and more realistic behaviour from an institutional risk standpoint.

4.2.3. Higher Moment Portfolios

When higher-order moments (skewness and kurtosis, where positive skewness ($\gamma_1 > 0$) implies upside potential; low excess kurtosis ($\gamma_2 \approx 0$) signals thinner tails), shown in Appendix 6, are incorporated, MBTE-Higher Moments achieves a Sharpe ratio above 1.2, outperforming both BL-based and combined models. This is expected: MBTE's optimization

is driven by benchmark-relative alignment, which benefits from smooth, benchmark-like distributions and avoids exposure to extreme tail risk.

The Traditional Mean–Variance model, though showing the highest absolute returns, also exhibits elevated volatility and kurtosis, signalling greater exposure to fat-tailed risk—an undesirable feature in risk-sensitive contexts.

The BL-MBTE-Higher Moments configuration remains almost identical to the BL-only variant in terms of return and volatility. Still, it benefits from a slightly lower tracking error, indicating that the MBTE constraint effectively controls deviation, even when the optimization considers skewness and kurtosis.

4.2.4. Discussion and Interpretation

Overall, the results reveal distinct behavioural patterns across the three optimization frameworks, as well as some unexpected underperformance from the combined (BL–MBTE) configuration.

First, the MBTE-only portfolios consistently demonstrate strong and stable outcomes across all optimization modes. They produce the highest Sharpe ratios and the lowest tracking errors, indicating that the benchmark-anchored design effectively captures broad market risk premia while limiting active risk. This is consistent with expectations of optimizing around benchmark co-movements, MBTE behaves like a sophisticated "core" strategy, generating robust performance even without subjective inputs.

In contrast, the Black–Litterman-only portfolios perform poorly in isolation. Their low or even negative Sharpe ratios suggest that the equilibrium priors dominate excessively when only a few, hard-coded investor views are provided. Since our implementation uses simplified views (two assets, moderate confidence), the model does not meaningfully deviate from equilibrium returns. This results in allocations that are too conservative, with weak return potential and relatively inefficient risk usage.

The combined BL–MBTE model was expected to achieve the best of both worlds, blending benchmark alignment (MBTE) with informed tilts from investor views (BL). However, the results show that it underperforms, particularly in Sharpe ratio and return metrics. This outcome suggests that the blending mechanism (via the α parameter) may have been overly restrictive, leading to portfolios that are "stuck in the middle", not aggressive enough to capture MBTE's benchmark-driven momentum, but also not sufficiently distinct to benefit from BL's subjective adjustments.

Intuitively, when the α parameter is set closer to 0.6–0.65, as in our implementation, the model heavily penalizes deviation from benchmark weights. This ties the portfolio closely to the MBTE structure, thereby diluting the incremental benefit of BL adjustments. Moreover, since the BL views were limited in scope, their influence on expected returns was minor compared to the magnitude of benchmark returns, further reducing the hybrid model's effectiveness.

Another factor could be model interaction noise: BL introduces prior-adjusted expected returns, while MBTE constrains allocation dispersion. When combined, these objectives may not align smoothly, especially if benchmark and BL priors conflict in direction or magnitude. This can lead to optimization trade-offs that slightly degrade efficiency, even if the approach is conceptually appealing.

In short, while the combined model demonstrates theoretical appeal, the empirical results presented here show that its practical benefit depends strongly on calibration; both in terms of α blending strength and the richness of input views. Without diverse or high-confidence BL signals, the hybrid approach risks converging to a suboptimal midpoint between two better-performing extremes.

4.3. Summary

Across all optimization configurations, the MBTE-only portfolios exhibit the most stable and risk-efficient performance, achieving the highest Sharpe ratios and lowest tracking errors. The Black–Litterman-only portfolios, constrained by limited and moderately confident views, underperformed, underscoring the model's reliance on robust, well-specified investor expectations. The combined BL–MBTE framework, while theoretically designed to balance active and passive traits, underdelivered in practice, suggesting that its blending parameter ($\alpha \approx 0.6$–0.65) may have been overly restrictive or that the conflicting objectives of view adjustment and benchmark tracking reduced efficiency.

Taken together, these findings suggest that MBTE provides a more resilient baseline structure in data-driven settings. At the same time, the BL–MBTE hybrid may require more granular calibration, such as, a more adaptive α parameter or a richer view matrix, to fully realize its theoretical advantages. Future refinements could explore dynamic weighting schemes, scenario-based view generation, or time-varying confidence levels to enhance responsiveness and improve real-world performance.

5. Limitations and Future Work

Both methods have distinct weaknesses: Black–Litterman is highly sensitive to how views, their confidences (Ω), and the prior scale (τ) are set poor calibration or using non–market-cap priors can produce unstable, concentrated tilts (e.g., outsized single-name weights), especially under long-only constraints and noisy estimates of μ/Σ. MBTE, by contrast, prioritizes benchmark alignment but can suppress alpha when γ or TE caps are tight; equal-weighted averaging across benchmarks may misrepresent policy importance, static caps aren't regime-aware, and the framework lacks a directional "view" layer. In short: BL can overreact to views and priors; MBTE can over-hug benchmarks and under-exploit opportunities choice and tuning should reflect mandate and regime.

5.1 Limitations

Despite demonstrating flexibility and interpretability, our framework presents several limitations. The model's reliance on heuristic parameters (τ, Ω, γ, and the blend factor α) introduces allocation sensitivity and increases the risk of over-concentration when underlying market signals are noisy. Moreover, the Black–Litterman prior is anchored on equal weights rather than true market-capitalization data, weakening the "market-implied" equilibrium baseline. The use of manually specified investor views and confidence levels also introduces potential bias, as subjective bullish or bearish tilts can distort portfolio weights toward certain securities.

Additionally, the optimization imposes long-only, fully invested constraints without accounting for turnover, transaction costs, or borrowing fees, leading to optimistic performance estimates. The multi-benchmark treatment currently applies equal weighting across all benchmarks, which overlooks their varying policy or economic significance. Finally, estimation errors in expected returns, covariances, and factor loadings, particularly under short rolling windows or shifting market regimes, can yield unstable weights, while static tracking error caps fail to adjust dynamically to volatility changes, potentially constraining performance inappropriately across different market states.

5.2 Future Work

Future extensions can address these limitations through both structural and methodological enhancements. The prior distribution should be replaced with free-float market capitalization weights, while parameters such as τ, Ω, γ, and α can be systematically cross-validated over rolling windows to improve stability. Benchmark importances (φ) could be learned endogenously rather than averaged equally, providing a more accurate reflection of their relative influence.

Additionally, the optimization framework can be extended to allow short-selling with L1/L2 regularization, transaction cost modeling, turnover constraints, and trade-to-target penalties. Tracking error (TE) caps could be made volatility-aware, scaling with benchmark variance,

and designed to apply asymmetric penalties when breached. To further stabilize exposures, ridge or elastic-net regressions could be incorporated to enforce factor neutrality across style, sector, or beta dimensions. Systematic view generation using a signal library (e.g., views derived from factor t-statistics or information ratios) would also enhance reproducibility and reduce subjective bias. Finally, adopting robust or shrinkage-based covariance estimators, resampled efficient frontiers, and more comprehensive evaluation methods, such as walk-forward validation, sub-period stress tests, and Brinson-style performance attribution, would strengthen both the robustness and interpretability of results.

6. References

Apichat, C. & Rujira, C. (2022). *Markowitz mean-variance portfolio optimization with predictive stock selection using machine learning*. Int. J. Financial Stud. 2022, 10(3), 64; https://doi.org/10.3390/ijfs10030064

Abou Rjaily, Natasha and Roncalli, Thierry and Xu, Jiali. (2024). *Risk Factor, Risk Premium and Black-Litterman Model*. *SSRN*: http://dx.doi.org/10.2139/ssrn.4976695

Beardsley, X. W., Field, B., & Xiao, M. (2012). *Mean-Variance-Skewness-Kurtosis Portfolio Optimization with Return and Liquidity*. Communications in Mathematical Finance, vol. 1, no.1, 2012, 13-49

Chen, S D. & Lim, A E. B. (2020). *A Generalized Black–Litterman Model. Operations research, 68(2), 381-410*. https://doi.org/10.1287/opre.2019.1893

Chen, W., Zhang, H., Mehlawat, M. K., & Jia, L. (2021). *Mean–variance portfolio optimization using machine learning-based stock price prediction*. *Applied Soft Computing, 100*, 106943. https://doi-org.libproxy.smu.edu.sg/10.1016/j.asoc.2020.106943

D Zancato (2023). *Portfolio Optimization: An Introduction to Higher Order Moments* https://hdl.handle.net/20.500.14247/10681

Fama, E. F., & French, K. R. (2015). *A five-factor asset pricing model*. Journal of Financial Economics, 116(1), 1–22. https://doi.org/10.1016/j.jfineco.2015.01.004

Liu, P., Natarajan, K., Teo, C.-P., Xu, Y., & Zheng, Z. (2025, September 5). *Tracking the best: A multi-benchmark approach to portfolio construction.* Singapore Management University, Singapore University of Technology and Design, National University of Singapore.

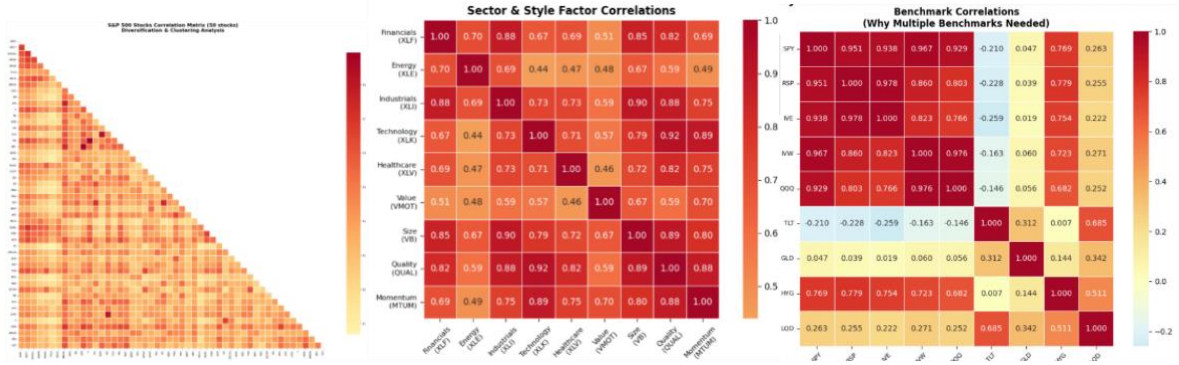
Markowitz, H. (1952). *Portfolio Selection*. *Journal of Finance, 7*(1), 77-91. https://doi.org/10.2307/2975974

Xu, Y., Zheng, Z., Natarajan, K., & Teo, C-P. (2014). *Tracking-error models for multiple benchmarks: Theory and empirical performance*. https://ink.library.smu.edu.sg/lkcsb_research/3774

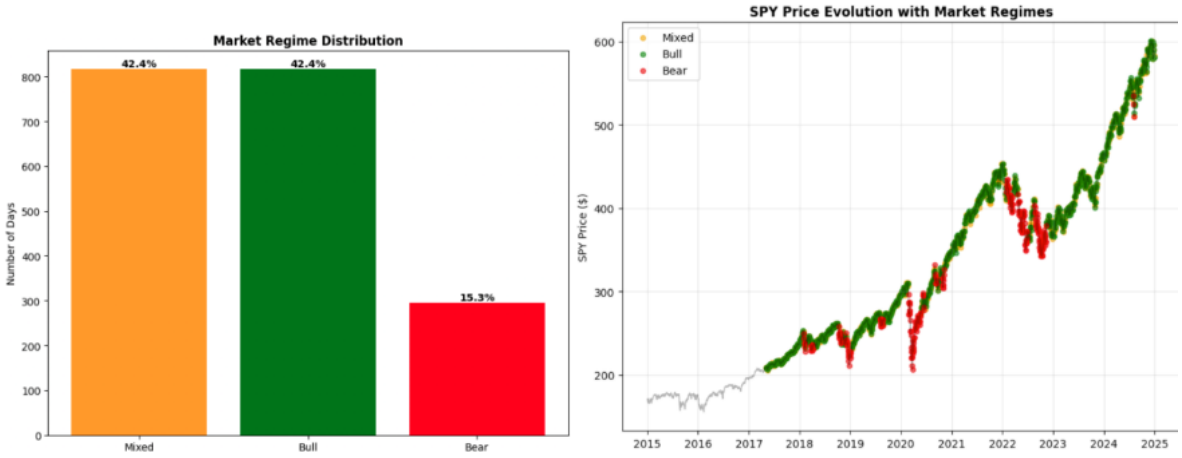
Z. Zhou, Z. Song, T. Ren and L. Yu. *Two-Stage Portfolio Optimization Integrating Optimal Sharp Ratio Measure and Ensemble Learning*. IEEE Access, vol. 11, pp. 1654-1670, 2023, doi: 10.1109/ACCESS.2022.3232281

7. Appendix

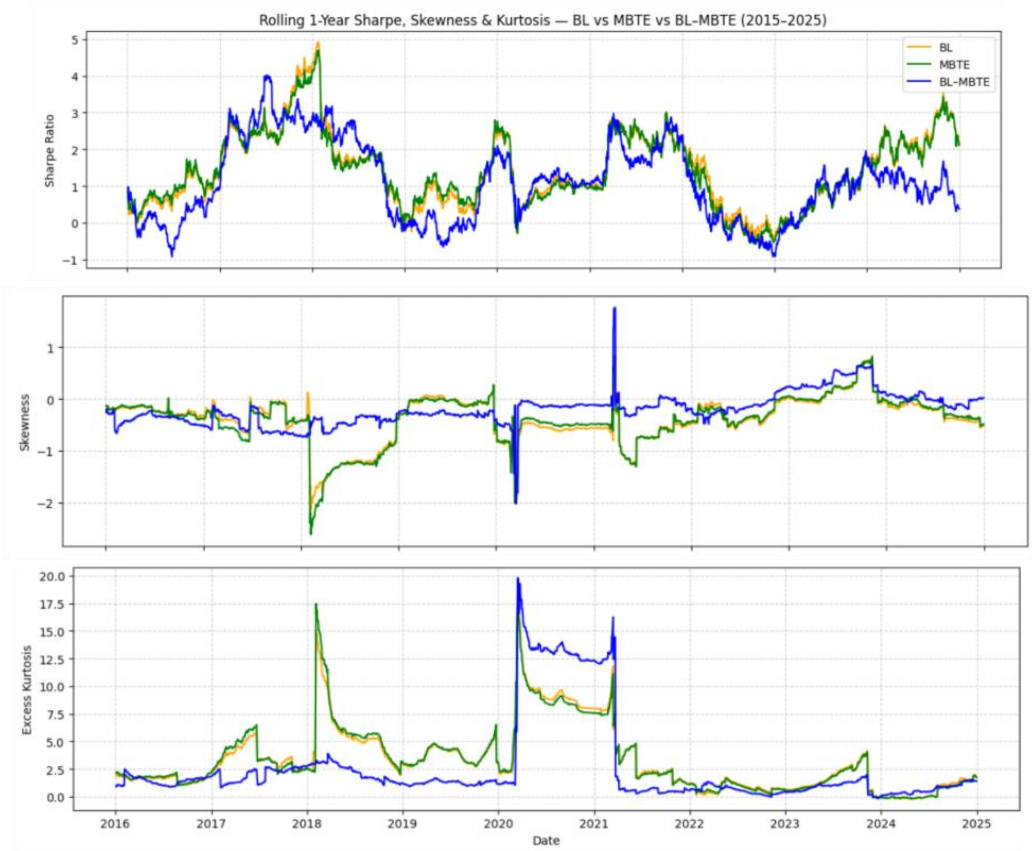
7.1. Appendix 1



7.2. Appendix 2



7.3. Appendix 3



7.4. Appendix 4

MEAN-VARIANCE PORTFOLIO METRICS:

	Expected_Return	Volatility	Sharpe_Ratio	Tracking_Error
BL_Only	2.323	1.797	-0.909	15.109
MBTE_Only	15.924	16.553	0.723	12.031
BL_MBTE	4.638	3.208	0.212	13.917

7.5. Appendix 5

MAXIMUM SHARPE RATIO PORTFOLIOS:

	Expected_Return	Volatility	Sharpe_Ratio	Tracking_Error
BL_Only	6.808	4.541	0.628	19.807
MBTE_Only	15.795	17.094	0.693	12.229
BL_MBTE	6.532	4.139	0.622	17.741

7.6. Appendix 6

```
HIGHER MOMENTS PORTFOLIO ANALYSIS:
--------------------------------------------------
                         Expected_Return  Volatility  Sharpe_Ratio  \
BL_Higher_Moments                  6.362       6.382         0.377
MBTE_Higher_Moments               43.849      33.072         1.206
BL_MBTE_Higher_Moments             6.347       6.354         0.376
Traditional_MV                    60.544      43.529         1.300

                         Tracking_Error  Skewness  Excess_Kurtosis
BL_Higher_Moments                29.549    -0.013            2.940
MBTE_Higher_Moments              27.629    -0.150            3.102
BL_MBTE_Higher_Moments           29.416    -0.015            2.940
Traditional_MV                   38.124     0.250            5.704
```

7.7. Appendix 7

Original Code:

```
# portfolio_bl_mbte_yf.py
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

from scipy.optimize import minimize
from scipy.stats import skew, kurtosis
import yfinance as yf
from datetime import datetime

# ------------------------------
# Ticker sets (from your brief)
# ------------------------------
SP500_TICKERS = [
    'AAPL','MSFT','GOOGL','AMZN','TSLA','META',"NVDA",'BRK-B','UNH','JNJ','JPM','V',
    'PG','XOM','HD','CVX','MA','BAC','ABBV','PFE','AVGO','COST','DIS','KO','MRK',
    'TMO','PEP','WMT','ABT','NFLX','ADBE','CRM','ACN','VZ','CMCSA','DHR','NKE',
    'TXN','BMY','NEE','QCOM','PM','RTX','UPS','LOW','T','HON','AMGN','IBM','CAT'
]

SECTOR_FACTORS = ['XLF','XLE','XLV',"XLI","XLK"]  # Financials, Energy, Industrials, Tech, Healthcare
STYLE_FACTORS  = ['VMOT','VB','QUAL','MTUM']      # Value, Size, Quality, Momentum

BENCHMARKS = {
    'SPY': 'S&P 500 (Cap-Weighted)',
    'RSP': 'S&P 500 (Equal-Weighted)',
    'IVE': 'S&P 500 Value',
    'IVW': 'S&P 500 Growth',
    'QQQ': 'NASDAQ 100',
    "TLT" : "TLT",
```

```
    "GLD":"GLD",
    "HYG": "HYG",
    "LQD": "LQD"
}

# ----------------------------
# Data utilities
# ----------------------------
def _dl_adj_close(tickers, start, end):
    """Download Adj Close panel for tickers and return a clean wide DataFrame."""
    if isinstance(tickers, (list, tuple, set)):
        tickers = list(tickers)
    data = yf.download(tickers, start=start, end=end, auto_adjust=False, progress=False)
    # yfinance returns columns either as single-level (when 1 ticker) or multi-level
    if isinstance(data.columns, pd.MultiIndex):
        adj = data['Close'].copy()
    else:
        adj = data[['Close']].rename(columns={'Close': tickers if isinstance(tickers, str) else tickers[0]})
    # Some tickers might not pull; drop all-null cols
    adj = adj.dropna(axis=1, how='all')
    return adj

def _pct_change(df):
    return df.sort_index().pct_change().dropna(how='all')

def load_market_data(start='2015-01-01', end=None):
    """
    Returns:
      assets_ret  : DataFrame (daily % returns) for 50 S&P tickers
      sector_ret  : DataFrame for sector ETFs
      style_ret   : DataFrame for style ETFs
      bench_ret   : DataFrame for benchmark ETFs
      rfr_annual  : float annualized risk-free (from ^IRX), e.g., 0.04
    """
    if end is None:
        end = datetime.today().strftime('%Y-%m-%d')

    # Assets
    prices_assets = _dl_adj_close(SP500_TICKERS, start, end)
    assets_ret = _pct_change(prices_assets).dropna(axis=1, how='any')  # keep only tickers with data

    # Sector + Style factors
    prices_sector = _dl_adj_close(SECTOR_FACTORS, start, end)
    sector_ret = _pct_change(prices_sector)

    prices_style = _dl_adj_close(STYLE_FACTORS, start, end)
    style_ret = _pct_change(prices_style)

    # Benchmarks
    prices_bench = _dl_adj_close(list(BENCHMARKS.keys()), start, end)
    bench_ret = _pct_change(prices_bench)

    # Align all on common dates
    common_idx = assets_ret.index
    for df in [sector_ret, style_ret, bench_ret]:
        common_idx = common_idx.intersection(df.index)

    assets_ret = assets_ret.loc[common_idx]
    sector_ret = sector_ret.loc[common_idx]
    style_ret  = style_ret.loc[common_idx]
    bench_ret  = bench_ret.loc[common_idx]

    # Risk-free from 13-week T-bill (^IRX) — Yahoo gives % yield. Convert to decimal.
    irx = yf.download('^IRX', start=start, end=end, progress=False)['Close'].dropna()
    # Smooth a bit to avoid a single-day blip; use last 60 trading days average
    if len(irx) >= 60:
```

```
        rfr_annual = float(irx.iloc[-60:].mean() / 100.0)
    else:
        rfr_annual = float(irx.iloc[-1] / 100.0)

    # Basic hygiene: remove any columns with too many NaNs (rare with these ETFs)
    min_valid = int(0.9 * len(common_idx))
    assets_ret = assets_ret.loc[:, assets_ret.count() >= min_valid]

    return assets_ret, sector_ret, style_ret, bench_ret, rfr_annual

# ----------------------------
# Optimizer classes (your originals, kept)
# ----------------------------
class PortfolioOptimizer:
    def __init__(self, returns_data, risk_free_rate=0.02):
        self.returns = returns_data
        self.risk_free_rate = risk_free_rate
        self.n_assets = len(returns_data.columns)
        self.asset_names = returns_data.columns.tolist()

        # Annualized stats
        self.mean_returns = returns_data.mean() * 252
        self.cov_matrix   = returns_data.cov() * 252
        self.corr_matrix  = returns_data.corr()

        # Higher moments (daily)
        self.skewness = returns_data.apply(skew)
        self.excess_kurtosis = returns_data.apply(lambda x: kurtosis(x, fisher=True))

        print(f"Initialized optimizer with {self.n_assets} assets")
        print(f"Sample period: {len(returns_data)} observations ({returns_data.index.min().date()} → {returns_data.index.max().date()})")

class BlackLittermanOptimizer(PortfolioOptimizer):
    def __init__(self, returns_data, market_caps=None, risk_aversion=3.0, tau=0.05, risk_free_rate=0.02):
        super().__init__(returns_data, risk_free_rate)
        self.risk_aversion = risk_aversion
        self.tau = tau

        if market_caps is None:
            self.market_weights = np.ones(self.n_assets) / self.n_assets
        else:
            self.market_weights = np.array(market_caps) / np.sum(market_caps)

        self.equilibrium_returns = self._calculate_equilibrium_returns()

    def _calculate_equilibrium_returns(self):
        return self.risk_aversion * np.dot(self.cov_matrix, self.market_weights)

    def update_views(self, P, Q, omega):
        self.P, self.Q, self.omega = P, Q, omega
        tau_cov = self.tau * self.cov_matrix

        M1 = np.linalg.inv(tau_cov)
        M2 = np.dot(P.T, np.dot(np.linalg.inv(omega), P))
        M3 = np.dot(np.linalg.inv(tau_cov), self.equilibrium_returns)
        M4 = np.dot(P.T, np.dot(np.linalg.inv(omega), Q))

        self.bl_returns = np.dot(np.linalg.inv(M1 + M2), M3 + M4)
        # NOTE: Many implementations use the market covariance for optimization.
        # We keep your original line but you may prefer: self.bl_cov = self.cov_matrix
        self.bl_cov = np.linalg.inv(M1 + M2)

        return self.bl_returns, self.bl_cov

class MBTEOptimizer(PortfolioOptimizer):
```

```
    def __init__(self, returns_data, benchmark_returns, risk_free_rate=0.02):
        super().__init__(returns_data, risk_free_rate)
        self.benchmarks = benchmark_returns if isinstance(benchmark_returns, pd.DataFrame) else benchmark_returns.to_frame()
        self.n_benchmarks = len(self.benchmarks.columns)
        self.benchmark_names = self.benchmarks.columns.tolist()
        self.benchmark_returns = self.benchmarks.mean() * 252
        self.benchmark_cov = self.benchmarks.cov() * 252
        print(f"MBTE initialized with {self.n_benchmarks} benchmarks: {', '.join(self.benchmark_names)}")

# ------------------------------
# Metrics & helpers
# ------------------------------
def portfolio_performance(weights, returns, cov_matrix, risk_free_rate=0.02):
    port_ret = float(np.dot(weights, returns))
    port_vol = float(np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights))))
    sharpe = (port_ret - risk_free_rate) / port_vol if port_vol > 0 else np.nan
    return port_ret, port_vol, sharpe

def tracking_error(weights, asset_returns_df, benchmark_returns):
    """
    Annualized TE. If benchmark_returns is DataFrame -> TE averaged across all benchmarks.
    """
    port_series = asset_returns_df.dot(weights)
    if isinstance(benchmark_returns, pd.DataFrame):
        tes = []
        for col in benchmark_returns.columns:
            te = (port_series - benchmark_returns[col]).std() * np.sqrt(252)
            tes.append(te)
        return float(np.mean(tes))
    else:
        return float((port_series - benchmark_returns).std() * np.sqrt(252))

# ------------------------------
# Analyses (now using real data)
# ------------------------------
def mean_variance_analysis(returns_data, benchmark_data, risk_free_rate):
    print("\n" + "="*60)
    print("PORTFOLIO A: MEAN-VARIANCE ANALYSIS (Real yfinance data)")
    print("="*60)

    asset_names = list(returns_data.columns)
    n_assets = len(asset_names)

    base_opt = PortfolioOptimizer(returns_data, risk_free_rate)
    bl_opt   = BlackLittermanOptimizer(returns_data, risk_free_rate=risk_free_rate)
    mbte_opt = MBTEOptimizer(returns_data, benchmark_data, risk_free_rate=risk_free_rate)

    # Example BL views (tweak as desired): overweight first asset, underweight a bond-proxy if present
    # Here we simply place views on two liquid mega-caps if available
    candidates = [t for t in ['AAPL','MSFT','JPM','XOM'] if t in asset_names]
    if len(candidates) >= 2:
        idx_a = asset_names.index(candidates[0])
        idx_b = asset_names.index(candidates[1])
    else:
        idx_a, idx_b = 0, 1

    P = np.zeros((2, n_assets))
    P[0, idx_a] = 1.0
    P[1, idx_b] = 1.0
    Q = np.array([0.02, -0.01])     # annualized excess returns view
    omega = np.diag([0.001, 0.001]) # view uncertainty
    bl_opt.update_views(P, Q, omega)

    def optimize_portfolio(expected_returns, cov_matrix, objective='min_vol', target_return=None):
        def obj(w):
            if objective == 'min_vol':
```

```
            return float(np.sqrt(np.dot(w.T, np.dot(cov_matrix, w))))
        elif objective == 'max_sharpe':
            r = float(np.dot(w, expected_returns))
            v = float(np.sqrt(np.dot(w.T, np.dot(cov_matrix, w))))
            return -(r - risk_free_rate) / v if v > 0 else 1e9
        elif objective == 'target_return':
            return float(np.sqrt(np.dot(w.T, np.dot(cov_matrix, w))))
        return 0.0

    cons = [{'type': 'eq', 'fun': lambda w: np.sum(w) - 1}]
    if objective == 'target_return' and target_return is not None:
        cons.append({'type': 'eq', 'fun': lambda w: np.dot(w, expected_returns) - target_return})
    bnds = tuple([(0, 1) for _ in range(n_assets)])

    res = minimize(obj, x0=np.ones(n_assets)/n_assets, method='SLSQP', bounds=bnds, constraints=cons)
    return res.x if res.success else np.ones(n_assets)/n_assets

portfolios = {}

# 1) BL-only (min vol)
bl_weights = optimize_portfolio(bl_opt.bl_returns, bl_opt.bl_cov, 'min_vol')
portfolios['BL_Only'] = {'weights': bl_weights, 'returns': bl_opt.bl_returns, 'cov': bl_opt.bl_cov}

# 2) MBTE-only: minimize average TE to all benchmarks
def mbte_objective(w):
    return tracking_error(w, returns_data, benchmark_data)

res_mbte = minimize(mbte_objective, x0=np.ones(n_assets)/n_assets, method='SLSQP',
                    bounds=tuple([(0,1)]*n_assets),
                    constraints=[{'type':'eq','fun':lambda w: np.sum(w)-1}])
portfolios['MBTE_Only'] = {'weights': res_mbte.x if res_mbte.success else np.ones(n_assets)/n_assets,
                           'returns': base_opt.mean_returns, 'cov': base_opt.cov_matrix}

# 3) BL + MBTE combo: utility minus multi-benchmark TE penalty
def combined_objective(w, alpha=0.6):
    bl_util = np.dot(w, bl_opt.bl_returns) - 0.5 * 3.0 * np.dot(w.T, np.dot(bl_opt.bl_cov, w))
    te_pen  = tracking_error(w, returns_data, benchmark_data)  # annualized
    return -(alpha * bl_util - (1 - alpha) * te_pen**2)

res_comb = minimize(lambda w: combined_objective(w, 0.6), x0=np.ones(n_assets)/n_assets, method='SLSQP',
                    bounds=tuple([(0,1)]*n_assets),
                    constraints=[{'type':'eq','fun':lambda w: np.sum(w)-1}])
portfolios['BL_MBTE'] = {'weights': res_comb.x if res_comb.success else np.ones(n_assets)/n_assets,
                         'returns': bl_opt.bl_returns, 'cov': bl_opt.bl_cov}

# 4) BL + Single Benchmark (for comparison, same as BL min-vol)
portfolios['BL_Single_Bench'] = {'weights': bl_weights, 'returns': bl_opt.bl_returns, 'cov': bl_opt.bl_cov}

# Metrics
results_df = pd.DataFrame(index=portfolios.keys())
for name, p in portfolios.items():
    w = p['weights']
    ret, vol, shrp = portfolio_performance(w, p['returns'], p['cov'], risk_free_rate)
    te = tracking_error(w, returns_data, benchmark_data)
    results_df.loc[name, 'Expected_Return'] = ret * 100
    results_df.loc[name, 'Volatility'] = vol * 100
    results_df.loc[name, 'Sharpe_Ratio'] = shrp
    results_df.loc[name, 'Tracking_Error'] = te * 100

print("\nMEAN-VARIANCE PORTFOLIO METRICS:")
print("-"*50)
print(results_df.round(3))

allocations_df = pd.DataFrame({k: v['weights'] for k,v in portfolios.items()}, index=asset_names)
print("\nPORTFOLIO ALLOCATIONS (%):")
print("-"*40)
```

```
    print((allocations_df * 100).round(1))

    # Simple plots
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))
    axes[0,0].scatter(results_df['Volatility'], results_df['Expected_Return'], s=100)
    for i, name in enumerate(results_df.index):
        axes[0,0].annotate(name, (results_df.iloc[i]['Volatility'], results_df.iloc[i]['Expected_Return']))
    axes[0,0].set_xlabel('Volatility (%)'); axes[0,0].set_ylabel('Expected Return (%)'); axes[0,0].set_title('Risk-Return'); axes[0,0].grid(True)

    (allocations_df.T*100).plot(kind='bar', ax=axes[0,1], width=0.8)
    axes[0,1].set_title('Portfolio Allocations (%)'); axes[0,1].legend(bbox_to_anchor=(1.05,1), loc='upper left')

    results_df['Sharpe_Ratio'].plot(kind='bar', ax=axes[1,0])
    axes[1,0].set_title('Sharpe Ratios')

    results_df['Tracking_Error'].plot(kind='bar', ax=axes[1,1], color='orange')
    axes[1,1].set_title('Tracking Error vs Multi-Benchmark'); axes[1,1].set_ylabel('TE (%)')

    plt.tight_layout(); plt.show()

    return portfolios, results_df, allocations_df

def max_sharpe_analysis(returns_data, benchmark_data, risk_free_rate):
    print("\n" + "="*60)
    print("PORTFOLIO B: MAXIMUM SHARPE RATIO ANALYSIS (Real yfinance data)")
    print("="*60)

    base_opt = PortfolioOptimizer(returns_data, risk_free_rate)
    bl_opt   = BlackLittermanOptimizer(returns_data, risk_free_rate=risk_free_rate)
    mbte_opt = MBTEOptimizer(returns_data, benchmark_data, risk_free_rate=risk_free_rate)

    # BL views (same as before)
    n_assets = bl_opt.n_assets
    P = np.zeros((2, n_assets)); P[0,0] = 1; P[1,1] = 1
    Q = np.array([0.02, -0.01]); omega = np.diag([0.001, 0.001])
    bl_opt.update_views(P, Q, omega)

    def maximize_sharpe(exp_ret, cov, rfr):
        def neg_sr(w):
            r = float(np.dot(w, exp_ret)); v = float(np.sqrt(np.dot(w.T, np.dot(cov, w))))
            return -(r - rfr)/v if v > 0 else 1e9
        cons = [{'type': 'eq', 'fun': lambda w: np.sum(w)-1}]
        bnds = tuple([(0,1)]*n_assets)
        res = minimize(neg_sr, x0=np.ones(n_assets)/n_assets, method='SLSQP', bounds=bnds, constraints=cons)
        return res.x if res.success else np.ones(n_assets)/n_assets

    max_sharpe_portfolios = {}
    max_sharpe_portfolios['BL_Only'] = maximize_sharpe(bl_opt.bl_returns, bl_opt.bl_cov, risk_free_rate)

    def constrained_sharpe_mbte(w):
        r = float(np.dot(w, base_opt.mean_returns)); v = float(np.sqrt(np.dot(w.T, np.dot(base_opt.cov_matrix, w))))
        return -(r - risk_free_rate)/v if v > 0 else 1e9

    def te_constraint(w, max_te=0.08):
        te = tracking_error(w, returns_data, benchmark_data)
        return max_te - te

    res_mbte = minimize(constrained_sharpe_mbte, x0=np.ones(n_assets)/n_assets, method='SLSQP',
                        bounds=tuple([(0,1)]*n_assets),
                        constraints=[{'type':'eq','fun':lambda w: np.sum(w)-1},
                                     {'type':'ineq','fun':lambda w: te_constraint(w, 0.08)}])
    max_sharpe_portfolios['MBTE_Only'] = res_mbte.x if res_mbte.success else np.ones(n_assets)/n_assets

    def combined_obj(w, alpha=0.6):
        bl_r = float(np.dot(w, bl_opt.bl_returns))
        bl_v = float(np.sqrt(np.dot(w.T, np.dot(bl_opt.bl_cov, w))))
```

```
        bl_sr = (bl_r - risk_free_rate)/bl_v if bl_v > 0 else 0.0
        te_pen = tracking_error(w, returns_data, benchmark_data)
        return -(alpha*bl_sr - (1-alpha)*te_pen)

    res_comb = minimize(lambda w: combined_obj(w, 0.6), x0=np.ones(n_assets)/n_assets, method='SLSQP',
                        bounds=tuple([(0,1)]*n_assets), constraints=[{'type':'eq','fun':lambda w: np.sum(w)-1}])
    max_sharpe_portfolios['BL_MBTE'] = res_comb.x if res_comb.success else np.ones(n_assets)/n_assets
    max_sharpe_portfolios['BL_Single_Bench'] = maximize_sharpe(bl_opt.bl_returns, bl_opt.bl_cov, risk_free_rate)

    sharpe_results = pd.DataFrame(index=max_sharpe_portfolios.keys())
    for name, w in max_sharpe_portfolios.items():
        if 'BL' in name:
            exp_ret, cov = bl_opt.bl_returns, bl_opt.bl_cov
        else:
            exp_ret, cov = base_opt.mean_returns, base_opt.cov_matrix
        ret, vol, shrp = portfolio_performance(w, exp_ret, cov, risk_free_rate)
        te = tracking_error(w, returns_data, benchmark_data)
        sharpe_results.loc[name, ['Expected_Return','Volatility','Sharpe_Ratio','Tracking_Error']] = [ret*100, vol*100, shrp, te*100]

    print("\nMAXIMUM SHARPE RATIO PORTFOLIOS:")
    print("-"*45); print(sharpe_results.round(3))

    baseline = sharpe_results.loc['BL_Only','Sharpe_Ratio']
    print("\nSHARPE RATIO IMPROVEMENTS:")
    print("-"*35)
    for name in sharpe_results.index:
        imp = sharpe_results.loc[name,'Sharpe_Ratio'] - baseline
        pct = (imp/baseline*100) if pd.notna(baseline) and baseline != 0 else np.nan
        print(f"{name}: {imp:+.3f} ({pct:+.1f}%)")

    fig, axes = plt.subplots(1, 3, figsize=(18, 6))
    sharpe_results['Sharpe_Ratio'].plot(kind='bar', ax=axes[0])
    axes[0].set_title('Maximum Sharpe Ratios')

    axes[1].scatter(sharpe_results['Volatility'], sharpe_results['Expected_Return'], s=150)
    for i, name in enumerate(sharpe_results.index):
        axes[1].annotate(name, (sharpe_results.iloc[i]['Volatility'], sharpe_results.iloc[i]['Expected_Return']))
    axes[1].set_xlabel('Volatility (%)'); axes[1].set_ylabel('Expected Return (%)'); axes[1].set_title('Risk-Return'); axes[1].grid(True)

    te_vs = sharpe_results[['Tracking_Error','Sharpe_Ratio']]
    axes[2].scatter(te_vs['Tracking_Error'], te_vs['Sharpe_Ratio'], s=150)
    for i, name in enumerate(te_vs.index):
        axes[2].annotate(name, (te_vs.iloc[i]['Tracking_Error'], te_vs.iloc[i]['Sharpe_Ratio']))
    axes[2].set_xlabel('Tracking Error (%)'); axes[2].set_ylabel('Sharpe Ratio'); axes[2].set_title('TE vs Sharpe'); axes[2].grid(True)

    plt.tight_layout(); plt.show()
    return max_sharpe_portfolios, sharpe_results

def higher_moments_analysis(returns_data, benchmark_data, risk_free_rate):
    print("\n" + "="*60)
    print("PORTFOLIO C: HIGHER MOMENTS ANALYSIS (Real yfinance data)")
    print("="*60)

    asset_names = list(returns_data.columns)
    base_opt = PortfolioOptimizer(returns_data, risk_free_rate)
    bl_opt   = BlackLittermanOptimizer(returns_data, risk_free_rate=risk_free_rate)
    mbte_opt = MBTEOptimizer(returns_data, benchmark_data, risk_free_rate=risk_free_rate)

    # Views (as before)
    n_assets = bl_opt.n_assets
    P = np.zeros((2, n_assets)); P[0,0]=1; P[1,1]=1
    Q = np.array([0.02, -0.01]); omega = np.diag([0.001, 0.001])
    bl_opt.update_views(P, Q, omega)

    def calculate_portfolio_moments(w, ret_df):
        port = ret_df.dot(w)
```

```
    return float(skew(port)), float(kurtosis(port, fisher=True))

def higher_moment_objective(w, exp_ret, cov, ret_df, skew_penalty=0.1, kurt_penalty=0.05):
    r = float(np.dot(w, exp_ret))
    var = float(np.dot(w.T, np.dot(cov, w)))
    mv_util = r - 0.5 * 3.0 * var
    port = ret_df.dot(w)
    ps = float(skew(port)); pk = float(kurtosis(port, fisher=True))
    skew_term = skew_penalty * min(0.0, ps)**2
    kurt_term = kurt_penalty * max(0.0, pk)**2
    return -(mv_util - skew_term - kurt_term)

n_assets = len(asset_names)

hm_ports = {}

# 1) BL with higher-moment penalties
res_bl_hm = minimize(lambda w: higher_moment_objective(w, bl_opt.bl_returns, bl_opt.bl_cov, returns_data.values),
                     x0=np.ones(n_assets)/n_assets, method='SLSQP',
                     bounds=tuple([(0,1)]*n_assets),
                     constraints=[{'type':'eq','fun':lambda w: np.sum(w)-1}])
hm_ports['BL_Higher_Moments'] = res_bl_hm.x if res_bl_hm.success else np.ones(n_assets)/n_assets

# 2) MBTE + higher-moments
def mbte_hm_obj(w):
    base_u = -higher_moment_objective(w, base_opt.mean_returns, base_opt.cov_matrix, returns_data.values)
    te_pen = tracking_error(w, returns_data, benchmark_data)
    return -(0.7*base_u - 0.3*te_pen**2)
res_mbte_hm = minimize(mbte_hm_obj, x0=np.ones(n_assets)/n_assets, method='SLSQP',
                       bounds=tuple([(0,1)]*n_assets),
                       constraints=[{'type':'eq','fun':lambda w: np.sum(w)-1}])
hm_ports['MBTE_Higher_Moments'] = res_mbte_hm.x if res_mbte_hm.success else np.ones(n_assets)/n_assets

# 3) Combined BL+MBTE + higher-moments
def comb_hm_obj(w, alpha=0.65):
    bl_u = -higher_moment_objective(w, bl_opt.bl_returns, bl_opt.bl_cov, returns_data.values, 0.15, 0.08)
    te_pen = tracking_error(w, returns_data, benchmark_data)
    return -(alpha*bl_u - (1-alpha)*te_pen**2)
res_comb_hm = minimize(lambda w: comb_hm_obj(w, 0.65), x0=np.ones(n_assets)/n_assets, method='SLSQP',
                       bounds=tuple([(0,1)]*n_assets),
                       constraints=[{'type':'eq','fun':lambda w: np.sum(w)-1}])
hm_ports['BL_MBTE_Higher_Moments'] = res_comb_hm.x if res_comb_hm.success else np.ones(n_assets)/n_assets

# 4) Traditional MV baseline
def mv_obj(w):
    r = float(np.dot(w, bl_opt.bl_returns)); v = float(np.dot(w.T, np.dot(bl_opt.bl_cov, w)))
    return -(r - 0.5 * 3.0 * v)
res_mv = minimize(mv_obj, x0=np.ones(n_assets)/n_assets, method='SLSQP',
                  bounds=tuple([(0,1)]*n_assets),
                  constraints=[{'type':'eq','fun':lambda w: np.sum(w)-1}])
hm_ports['Traditional_MV'] = res_mv.x if res_mv.success else np.ones(n_assets)/n_assets

# Metrics
hm_results = pd.DataFrame(index=hm_ports.keys())
for name, w in hm_ports.items():
    exp_ret, cov = (bl_opt.bl_returns, bl_opt.bl_cov) if ('BL' in name and name!='Traditional_MV') else (base_opt.mean_returns,
base_opt.cov_matrix)
    ret, vol, shrp = portfolio_performance(w, exp_ret, cov, risk_free_rate)
    te = tracking_error(w, returns_data, benchmark_data)
    ps, pk = calculate_portfolio_moments(w, returns_data)
    hm_results.loc[name, ['Expected_Return','Volatility','Sharpe_Ratio','Tracking_Error','Skewness','Excess_Kurtosis']] = \
        [ret*100, vol*100, shrp, te*100, ps, pk]

print("\nHIGHER MOMENTS PORTFOLIO ANALYSIS:")
print("-"*50); print(hm_results.round(3))
```

```
    hm_alloc = pd.DataFrame({k:v for k,v in hm_ports.items()}, index=asset_names)
    print("\nHIGHER MOMENTS PORTFOLIO ALLOCATIONS (%):")
    print("-"*50); print((hm_alloc*100).round(1))

    # Quick visuals
    fig, axes = plt.subplots(2, 3, figsize=(20, 12))
    bubble = np.abs(hm_results['Skewness'])*300 + 50
    sc = axes[0,0].scatter(hm_results['Volatility'], hm_results['Expected_Return'], s=bubble, alpha=0.7, c=hm_results['Excess_Kurtosis'], cmap='RdYlBu')
    for i, name in enumerate(hm_results.index):
        axes[0,0].annotate(name, (hm_results.iloc[i]['Volatility'], hm_results.iloc[i]['Expected_Return']), xytext=(5,5), textcoords='offset points', fontsize=8)
    axes[0,0].set_xlabel('Volatility (%)'); axes[0,0].set_ylabel('Expected Return (%)'); axes[0,0].set_title('Risk-Return (size=|skew|, color=kurt)'); plt.colorbar(sc, ax=axes[0,0])

    hm_results['Skewness'].plot(kind='bar', ax=axes[0,1]); axes[0,1].set_title('Portfolio Skewness'); axes[0,1].axhline(0, color='k', ls='--', alpha=0.4)
    hm_results['Excess_Kurtosis'].plot(kind='bar', ax=axes[0,2]); axes[0,2].set_title('Portfolio Excess Kurtosis'); axes[0,2].axhline(0, color='k', ls='--', alpha=0.4)

    (hm_alloc.T*100).plot(kind='bar', ax=axes[1,0], width=0.8); axes[1,0].set_title('Allocations (%)'); axes[1,0].legend(bbox_to_anchor=(1.05,1), loc='upper left')

    axes[1,1].scatter(hm_results['Skewness'], hm_results['Sharpe_Ratio'], s=100, alpha=0.7)
    for i, name in enumerate(hm_results.index):
        axes[1,1].annotate(name, (hm_results.iloc[i]['Skewness'], hm_results.iloc[i]['Sharpe_Ratio']), xytext=(5,5), textcoords='offset points', fontsize=8)
    axes[1,1].set_xlabel('Skewness'); axes[1,1].set_ylabel('Sharpe'); axes[1,1].set_title('Sharpe vs Skew'); axes[1,1].grid(True, alpha=0.3)

    # (Optional) omit higher-moment frontier for runtime brevity

    plt.tight_layout(); plt.show()

    return hm_ports, hm_results, hm_alloc

# ------------------------------
# Master runner
# ------------------------------
def run_complete_analysis(start='2015-01-01', end=None):
    print("COMPREHENSIVE PORTFOLIO OPTIMIZATION ANALYSIS (yfinance)")
    print("="*80)

    assets_ret, sector_ret, style_ret, bench_ret, rfr_annual = load_market_data(start, end)

    print(f"\nRisk-free (13w T-Bill, annualized): {rfr_annual:.2%}")
    print(f"Benchmarks loaded: {', '.join(bench_ret.columns)}")
    print(f"Assets loaded: {len(assets_ret.columns)} tickers")

    mv_portfolios, mv_results, mv_alloc = mean_variance_analysis(assets_ret, bench_ret, rfr_annual)
    sharpe_ports, sharpe_results = max_sharpe_analysis(assets_ret, bench_ret, rfr_annual)
    hm_ports, hm_results, hm_alloc = higher_moments_analysis(assets_ret, bench_ret, rfr_annual)

    print("\n" + "="*80)
    print("SUMMARY COMPARISON ACROSS ALL METHODS")
    print("="*80)

    best_mv = mv_results.loc[mv_results['Sharpe_Ratio'].idxmax()]
    best_sh = sharpe_results.loc[sharpe_results['Sharpe_Ratio'].idxmax()]
    best_hm = hm_results.loc[hm_results['Sharpe_Ratio'].idxmax()]

    summary = pd.DataFrame({
        'Best_MeanVariance': best_mv,
        'Best_MaxSharpe': best_sh,
        'Best_HigherMoments': best_hm
    }).T
```

```
    cols = ['Expected_Return','Volatility','Sharpe_Ratio','Tracking_Error']
    print("\nBEST PORTFOLIO FROM EACH METHOD:")
    print("-"*45); print(summary[cols].round(3))

    return {
        'mean_variance': (mv_portfolios, mv_results, mv_alloc),
        'max_sharpe': (sharpe_ports, sharpe_results),
        'higher_moments': (hm_ports, hm_results, hm_alloc),
        'risk_free_annual': rfr_annual
    }

if __name__ == "__main__":
    results = run_complete_analysis(start='2015-01-01', end=None)
```

New Updated Code:

```
"""
FastAPI API for running the Black–Litterman + Multi-Benchmark Tracking Error (MBTE)
portfolio analyses on live yfinance data, including a principled Hybrid BL+MBTE solver.

Usage
-----
1) Install deps (Python 3.10+ recommended):
  pip install fastapi uvicorn "pydantic<3" yfinance numpy pandas scipy

2) Run the server:
  uvicorn api_portfolio_bl_mbte:app --host 0.0.0.0 --port 8000 --reload

3) Open interactive docs:
  http://localhost:8000/docs

POST /analyze with JSON body like:
{
 "sp500_tickers": ["AAPL", "MSFT", "GOOGL", "AMZN", "TSLA", "META", "NVDA", "BRK-B", "UNH", "JNJ"],
 "sector_factors": ["XLF", "XLE", "XLV", "XLI", "XLK"],
 "style_factors": ["VMOT", "VB", "QUAL", "MTUM"],
 "benchmarks": {"SPY": "S&P 500 (Cap-Weighted)", "RSP": "S&P 500 (Equal-Weighted)", "IVE": "S&P 500 Value", "IVW": "S&P 500
Growth", "QQQ": "NASDAQ 100"},
 "start": "2015-01-01",
 "end": null,
 "which": ["mean_variance", "max_sharpe", "higher_moments", "hybrid"],
 "hybrid": {
   "bench_weight_mode": "anti_corr",
   "gamma": 1.25,
   "risk_aversion": 3.0,
   "tau": 0.05,
   "te_caps": {"SPY": 0.08, "QQQ": 0.10},
   "delta_factor_penalty": 0.15,
   "factor_target": [0,0,0,0,0, 0,0,0,0]
 }
}

Response contains metrics and chart-ready data (no images).
"""

from __future__ import annotations

import warnings
from datetime import datetime
from typing import Any, Dict, List, Optional, Union

warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
from pydantic import BaseModel, Field
from fastapi import FastAPI, HTTPException
from fastapi.responses import JSONResponse
from fastapi.middleware.cors import CORSMiddleware
from scipy.optimize import minimize
from scipy.stats import skew, kurtosis
import yfinance as yf

# ----------------------------
# Helpers (plots & JSON)
# ----------------------------

def df_to_split_json(df: pd.DataFrame) -> Dict[str, Any]:
   # Safer JSON-friendly format
```

```
    return df.reset_index().to_dict(orient="records")


# ----------------------------
# Data utilities
# ----------------------------

def _dl_adj_close(tickers: List[str], start: str, end: Optional[str]) -> pd.DataFrame:
    """Download Close prices for tickers and return a clean wide DataFrame."""
    if isinstance(tickers, (list, tuple, set)):
        tickers = list(tickers)
    if len(tickers) == 0:
        return pd.DataFrame()
    data = yf.download(tickers, start=start, end=end, auto_adjust=False, progress=False)
    if data.empty:
        return pd.DataFrame()
    # yfinance returns MultiIndex if multiple tickers
    if isinstance(data.columns, pd.MultiIndex):
        if "Close" in data.columns.get_level_values(0):
            adj = data["Close"].copy()
        else:
            # Some providers nest differently; fallback to last level named Close
            try:
                adj = data.xs("Close", axis=1, level=0, drop_level=False)
            except Exception:
                # last resort: pick last level
                adj = data.iloc[:, data.columns.get_level_values(-1) == "Close"]
                if isinstance(adj.columns, pd.MultiIndex):
                    adj.columns = adj.columns.get_level_values(-1)
    else:
        # Single ticker; unify into wide format
        name = tickers[0] if isinstance(tickers, list) and len(tickers) == 1 else "Close"
        adj = data[["Close"]].rename(columns={"Close": name})
    adj = adj.dropna(axis=1, how="all")
    return adj.dropna(how="all")


def _pct_change(df: pd.DataFrame) -> pd.DataFrame:
    return df.sort_index().pct_change().dropna(how="all")


def load_market_data(
    sp500_tickers: List[str],
    sector_factors: List[str],
    style_factors: List[str],
    benchmarks: Union[Dict[str, str], List[str]],
    start: str = "2015-01-01",
    end: Optional[str] = None,
):
    """
    Returns:
      assets_ret  : DataFrame (daily % returns) for S&P tickers
      sector_ret  : DataFrame for sector ETFs
      style_ret   : DataFrame for style ETFs
      bench_ret   : DataFrame for benchmark ETFs
      rfr_annual  : float annualized risk-free (from ^IRX), e.g., 0.04
    """
    if end is None:
        end = datetime.today().strftime("%Y-%m-%d")

    # Assets
    prices_assets = _dl_adj_close(sp500_tickers, start, end)
    assets_ret = _pct_change(prices_assets).dropna(axis=1, how="any")

    # Sector + Style factors
    prices_sector = _dl_adj_close(sector_factors, start, end)
```

```
    sector_ret = _pct_change(prices_sector)

    prices_style = _dl_adj_close(style_factors, start, end)
    style_ret = _pct_change(prices_style)

    # Benchmarks (accept dict or list)
    bench_list = list(benchmarks.keys()) if isinstance(benchmarks, dict) else list(benchmarks)
    prices_bench = _dl_adj_close(bench_list, start, end)
    bench_ret = _pct_change(prices_bench)

    # Align all on common dates
    common_idx = assets_ret.index
    for df in [sector_ret, style_ret, bench_ret]:
        common_idx = common_idx.intersection(df.index)

    assets_ret = assets_ret.loc[common_idx]
    sector_ret = sector_ret.loc[common_idx]
    style_ret = style_ret.loc[common_idx]
    bench_ret = bench_ret.loc[common_idx]

    # Risk-free from 13-week T-bill (^IRX) — Yahoo gives % yield. Convert to decimal.
    irx = yf.download("^IRX", start=start, end=end, progress=False)["Close"].dropna()
    if len(irx) == 0:
        rfr_annual = 0.02
    elif len(irx) >= 60:
        rfr_annual = float(irx.iloc[-60:].mean() / 100.0)
    else:
        rfr_annual = float(irx.iloc[-1] / 100.0)

    # Basic hygiene: remove any columns with too many NaNs (rare with these ETFs)
    min_valid = int(0.9 * len(common_idx)) if len(common_idx) > 0 else 0
    if min_valid > 0:
        assets_ret = assets_ret.loc[:, assets_ret.count() >= min_valid]

    return assets_ret, sector_ret, style_ret, bench_ret, rfr_annual


# ----------------------------
# Optimizer base classes
# ----------------------------

class PortfolioOptimizer:
    def __init__(self, returns_data: pd.DataFrame, risk_free_rate: float = 0.02):
        if returns_data is None or returns_data.empty:
            raise ValueError("returns_data is empty; check tickers/date range.")
        self.returns = returns_data
        self.risk_free_rate = risk_free_rate
        self.n_assets = len(returns_data.columns)
        self.asset_names = returns_data.columns.tolist()

        # Annualized stats
        self.mean_returns = returns_data.mean() * 252
        self.cov_matrix = returns_data.cov() * 252
        self.corr_matrix = returns_data.corr()

        # Higher moments (daily)
        self.skewness = returns_data.apply(skew)
        self.excess_kurtosis = returns_data.apply(lambda x: kurtosis(x, fisher=True))


class BlackLittermanOptimizer(PortfolioOptimizer):
    def __init__(self, returns_data: pd.DataFrame, market_caps: Optional[List[float]] = None,
                 risk_aversion: float = 3.0, tau: float = 0.05, risk_free_rate: float = 0.02):
        super().__init__(returns_data, risk_free_rate)
        self.risk_aversion = risk_aversion
        self.tau = tau
```

```
        if market_caps is None:
            self.market_weights = np.ones(self.n_assets) / self.n_assets
        else:
            self.market_weights = np.array(market_caps) / np.sum(market_caps)

        self.equilibrium_returns = self._calculate_equilibrium_returns()

    def _calculate_equilibrium_returns(self):
        cov = np.asarray(self.cov_matrix.values)
        return self.risk_aversion * np.dot(cov, self.market_weights)

    def update_views(self, P: np.ndarray, Q: np.ndarray, omega: np.ndarray):
        self.P, self.Q, self.omega = P, Q, omega
        cov = np.asarray(self.cov_matrix.values)
        tau_cov = self.tau * cov

        M1 = np.linalg.inv(tau_cov)
        M2 = np.dot(P.T, np.dot(np.linalg.inv(omega), P))
        M3 = np.dot(np.linalg.inv(tau_cov), self.equilibrium_returns)
        M4 = np.dot(P.T, np.dot(np.linalg.inv(omega), Q))

        inv_total = np.linalg.inv(M1 + M2)
        self.bl_returns = np.dot(inv_total, M3 + M4)
        # For BL covariance, a pragmatic choice is to keep original market covariance
        self.bl_cov = cov.copy()
        return self.bl_returns, self.bl_cov


class MBTEOptimizer(PortfolioOptimizer):
    def __init__(self, returns_data: pd.DataFrame, benchmark_returns: Union[pd.DataFrame, pd.Series], risk_free_rate: float = 0.02):
        super().__init__(returns_data, risk_free_rate)
        self.benchmarks = benchmark_returns if isinstance(benchmark_returns, pd.DataFrame) else benchmark_returns.to_frame()
        self.n_benchmarks = len(self.benchmarks.columns)
        self.benchmark_names = self.benchmarks.columns.tolist()
        self.benchmark_returns = self.benchmarks.mean() * 252
        self.benchmark_cov = self.benchmarks.cov() * 252


# ----------------------------
# Metrics & helpers
# ----------------------------

def portfolio_performance(weights: np.ndarray, returns: np.ndarray | pd.Series, cov_matrix: np.ndarray | pd.DataFrame, risk_free_rate: float
= 0.02):
    ret_vec = np.asarray(returns)
    cov = np.asarray(cov_matrix)
    w = np.asarray(weights)
    port_ret = float(np.dot(w, ret_vec))
    port_vol = float(np.sqrt(np.dot(w.T, np.dot(cov, w))))
    sharpe = (port_ret - risk_free_rate) / port_vol if port_vol > 0 else np.nan
    return port_ret, port_vol, sharpe


def tracking_error(weights: np.ndarray, asset_returns_df: pd.DataFrame, benchmark_returns: Union[pd.DataFrame, pd.Series]) -> float:
    """Annualized TE. If benchmark_returns is DataFrame -> TE averaged across all benchmarks."""
    port_series = asset_returns_df.dot(weights)
    if isinstance(benchmark_returns, pd.DataFrame):
        tes = []
        for col in benchmark_returns.columns:
            te = (port_series - benchmark_returns[col]).std() * np.sqrt(252)
            tes.append(te)
        return float(np.mean(tes))
    else:
        return float((port_series - benchmark_returns).std() * np.sqrt(252))
```

```
# ----------------------------
# Analyses (original three)
# ----------------------------

def mean_variance_analysis(
    returns_data: pd.DataFrame,
    benchmark_data: pd.DataFrame,
    risk_free_rate: float,
    blend_alpha: float = 0.6,
    te_gamma: float = 1.0,
):
    asset_names = list(returns_data.columns)
    n_assets = len(asset_names)

    base_opt = PortfolioOptimizer(returns_data, risk_free_rate)
    bl_opt = BlackLittermanOptimizer(returns_data, risk_free_rate=risk_free_rate)
    _ = MBTEOptimizer(returns_data, benchmark_data, risk_free_rate=risk_free_rate)

    # Example BL views: overweight first asset, underweight second (if exist)
    idx_a, idx_b = 0, 1 if n_assets > 1 else (0, 0)
    P = np.zeros((2, n_assets))
    P[0, idx_a] = 1.0
    P[1, idx_b] = 1.0
    Q = np.array([0.02, -0.01])
    omega = np.diag([0.001, 0.001])
    bl_opt.update_views(P, Q, omega)

    def optimize_portfolio(expected_returns, cov_matrix, objective="min_vol", target_return=None):
        def obj(w):
            cov = np.asarray(cov_matrix)
            if objective == "min_vol":
                return float(np.sqrt(np.dot(w.T, np.dot(cov, w))))
            elif objective == "max_sharpe":
                r = float(np.dot(w, expected_returns))
                v = float(np.sqrt(np.dot(w.T, np.dot(cov, w))))
                return -(r - risk_free_rate) / v if v > 0 else 1e9
            elif objective == "target_return" and target_return is not None:
                return float(np.sqrt(np.dot(w.T, np.dot(cov, w))))
            return 0.0

        cons = [{"type": "eq", "fun": lambda w: np.sum(w) - 1}]
        if objective == "target_return" and target_return is not None:
            cons.append({"type": "eq", "fun": lambda w: np.dot(w, expected_returns) - target_return})
        bnds = tuple([(0, 1) for _ in range(n_assets)])

        res = minimize(obj, x0=np.ones(n_assets) / n_assets, method="SLSQP", bounds=bnds, constraints=cons)
        return res.x if res.success else np.ones(n_assets) / n_assets

    portfolios: Dict[str, Dict[str, Any]] = {}

    # 1) BL-only (min vol)
    bl_weights = optimize_portfolio(bl_opt.bl_returns, bl_opt.bl_cov, "min_vol")
    portfolios["BL_Only"] = {"weights": bl_weights, "returns": bl_opt.bl_returns, "cov": bl_opt.bl_cov}

    # 2) MBTE-only: minimize average TE to all benchmarks
    def mbte_objective(w):
        return tracking_error(w, returns_data, benchmark_data)

    res_mbte = minimize(mbte_objective, x0=np.ones(n_assets) / n_assets, method="SLSQP",
                        bounds=tuple([(0, 1)] * n_assets), constraints=[{"type": "eq", "fun": lambda w: np.sum(w) - 1}])
    portfolios["MBTE_Only"] = {
        "weights": res_mbte.x if res_mbte.success else np.ones(n_assets) / n_assets,
        "returns": base_opt.mean_returns.values,
        "cov": base_opt.cov_matrix.values,
    }
```

```
# 3) BL + MBTE combo: utility minus multi-benchmark TE penalty
alpha = float(np.clip(blend_alpha, 0.0, 1.0))
gamma = max(float(te_gamma), 0.0)

def combined_objective(w):
    cov = np.asarray(bl_opt.bl_cov)
    bl_util = np.dot(w, bl_opt.bl_returns) - 0.5 * bl_opt.risk_aversion * np.dot(w.T, np.dot(cov, w))
    te_pen = tracking_error(w, returns_data, benchmark_data) ** 2  # annualized squared TE
    score = alpha * bl_util - (1.0 - alpha) * gamma * te_pen
    return -score

res_comb = minimize(combined_objective, x0=np.ones(n_assets) / n_assets, method="SLSQP",
                    bounds=tuple([(0, 1)] * n_assets), constraints=[{"type": "eq", "fun": lambda w: np.sum(w) - 1}])
portfolios["BL_MBTE"] = {"weights": res_comb.x if res_comb.success else np.ones(n_assets) / n_assets,
                         "returns": bl_opt.bl_returns, "cov": bl_opt.bl_cov}

# 4) BL + Single Benchmark (placeholder = BL min-vol)
portfolios["BL_Single_Bench"] = {"weights": bl_weights, "returns": bl_opt.bl_returns, "cov": bl_opt.bl_cov}

# Metrics
results_df = pd.DataFrame(index=portfolios.keys())
for name, p in portfolios.items():
    w = p["weights"]
    ret, vol, shrp = portfolio_performance(w, p["returns"], p["cov"], risk_free_rate)
    te = tracking_error(w, returns_data, benchmark_data)
    results_df.loc[name, "Expected_Return"] = ret * 100
    results_df.loc[name, "Volatility"] = vol * 100
    results_df.loc[name, "Sharpe_Ratio"] = shrp
    results_df.loc[name, "Tracking_Error"] = te * 100

allocations_df = pd.DataFrame({k: v["weights"] for k, v in portfolios.items()}, index=asset_names)

charts = {
    "risk_return": {
        "points": [
            {"name": name,
             "x": float(results_df.loc[name, "Volatility"]),
             "y": float(results_df.loc[name, "Expected_Return"])}
            for name in results_df.index.tolist()
        ],
        "x_label": "Volatility (%)",
        "y_label": "Expected Return (%)"
    },
    "allocations": {
        "assets": asset_names,
        "series": [
            {"name": name, "values": [float(v) for v in portfolios[name]["weights"].tolist()]}
            for name in portfolios.keys()
        ]
    },
    "sharpe_bar": {
        "labels": results_df.index.tolist(),
        "values": [float(v) for v in results_df["Sharpe_Ratio"].tolist()]
    },
    "te_bar": {
        "labels": results_df.index.tolist(),
        "values": [float(v) for v in results_df["Tracking_Error"].tolist()]
    }
}

return {
    "results": df_to_split_json(results_df.round(4)),
    "allocations": df_to_split_json((allocations_df * 100).round(2)),
    "charts": charts,
}
```

```
def max_sharpe_analysis(
    returns_data: pd.DataFrame,
    benchmark_data: pd.DataFrame,
    risk_free_rate: float,
    blend_alpha: float = 0.6,
    te_gamma: float = 1.0,
):
    bl_opt = BlackLittermanOptimizer(returns_data, risk_free_rate=risk_free_rate)
    base_opt = PortfolioOptimizer(returns_data, risk_free_rate)
    _ = MBTEOptimizer(returns_data, benchmark_data, risk_free_rate=risk_free_rate)

    n_assets = bl_opt.n_assets
    P = np.zeros((2, n_assets)); P[0, 0] = 1; P[1, 1] = 1
    Q = np.array([0.02, -0.01]); omega = np.diag([0.001, 0.001])
    bl_opt.update_views(P, Q, omega)

    def maximize_sharpe(exp_ret, cov, rfr):
        def neg_sr(w):
            covm = np.asarray(cov)
            r = float(np.dot(w, exp_ret)); v = float(np.sqrt(np.dot(w.T, np.dot(covm, w))))
            return -(r - rfr) / v if v > 0 else 1e9
        cons = [{"type": "eq", "fun": lambda w: np.sum(w) - 1}]
        bnds = tuple([(0, 1)] * n_assets)
        res = minimize(neg_sr, x0=np.ones(n_assets) / n_assets, method="SLSQP", bounds=bnds, constraints=cons)
        return res.x if res.success else np.ones(n_assets) / n_assets

    max_sharpe_portfolios: Dict[str, np.ndarray] = {}
    max_sharpe_portfolios["BL_Only"] = maximize_sharpe(bl_opt.bl_returns, bl_opt.bl_cov, risk_free_rate)

    def constrained_sharpe_mbte(w):
        covm = np.asarray(base_opt.cov_matrix.values)
        r = float(np.dot(w, base_opt.mean_returns.values)); v = float(np.sqrt(np.dot(w.T, np.dot(covm, w))))
        return -(r - risk_free_rate) / v if v > 0 else 1e9

    def te_constraint(w, max_te=0.08):
        te = tracking_error(w, returns_data, benchmark_data)
        return max_te - te

    res_mbte = minimize(constrained_sharpe_mbte, x0=np.ones(n_assets) / n_assets, method="SLSQP",
                        bounds=tuple([(0, 1)] * n_assets),
                        constraints=[{"type": "eq", "fun": lambda w: np.sum(w) - 1},
                                     {"type": "ineq", "fun": lambda w: te_constraint(w, 0.08)}])
    max_sharpe_portfolios["MBTE_Only"] = res_mbte.x if res_mbte.success else np.ones(n_assets) / n_assets

    alpha = float(np.clip(blend_alpha, 0.0, 1.0))
    gamma = max(float(te_gamma), 0.0)

    def combined_obj(w):
        covm = np.asarray(bl_opt.bl_cov)
        bl_r = float(np.dot(w, bl_opt.bl_returns))
        bl_v = float(np.sqrt(np.dot(w.T, np.dot(covm, w))))
        bl_sr = (bl_r - risk_free_rate) / bl_v if bl_v > 0 else 0.0
        te_pen = tracking_error(w, returns_data, benchmark_data)
        score = alpha * bl_sr - (1.0 - alpha) * gamma * te_pen
        return -score

    res_comb = minimize(combined_obj, x0=np.ones(n_assets) / n_assets, method="SLSQP",
                        bounds=tuple([(0, 1)] * n_assets), constraints=[{"type": "eq", "fun": lambda w: np.sum(w) - 1}])
    max_sharpe_portfolios["BL_MBTE"] = res_comb.x if res_comb.success else np.ones(n_assets) / n_assets
    max_sharpe_portfolios["BL_Single_Bench"] = maximize_sharpe(bl_opt.bl_returns, bl_opt.bl_cov, risk_free_rate)

    sharpe_results = pd.DataFrame(index=max_sharpe_portfolios.keys())
    for name, w in max_sharpe_portfolios.items():
        if "BL" in name:
```

```
        exp_ret, cov = bl_opt.bl_returns, bl_opt.bl_cov
    else:
        exp_ret, cov = base_opt.mean_returns.values, base_opt.cov_matrix.values
    ret, vol, shrp = portfolio_performance(w, exp_ret, cov, risk_free_rate)
    te = tracking_error(w, returns_data, benchmark_data)
    sharpe_results.loc[name, ["Expected_Return", "Volatility", "Sharpe_Ratio", "Tracking_Error"]] = [ret * 100, vol * 100, shrp, te * 100]

  charts = {
    "sharpe_bar": {
      "labels": sharpe_results.index.tolist(),
      "values": [float(v) for v in sharpe_results["Sharpe_Ratio"].tolist()]
    },
    "risk_return": {
      "points": [
        {"name": name,
         "x": float(sharpe_results.loc[name, "Volatility"]),
         "y": float(sharpe_results.loc[name, "Expected_Return"])}
        for name in sharpe_results.index.tolist()
      ],
      "x_label": "Volatility (%)",
      "y_label": "Expected Return (%)"
    },
    "te_vs_sharpe": {
      "points": [
        {"name": name,
         "x": float(sharpe_results.loc[name, "Tracking_Error"]),
         "y": float(sharpe_results.loc[name, "Sharpe_Ratio"])}
        for name in sharpe_results.index.tolist()
      ],
      "x_label": "Tracking Error (%)",
      "y_label": "Sharpe Ratio"
    }
  }

  return {
    "results": df_to_split_json(sharpe_results.round(4)),
    "charts": charts,
  }


def higher_moments_analysis(
  returns_data: pd.DataFrame,
  benchmark_data: pd.DataFrame,
  risk_free_rate: float,
  blend_alpha: float = 0.65,
  te_gamma: float = 1.0,
):
  asset_names = list(returns_data.columns)
  base_opt = PortfolioOptimizer(returns_data, risk_free_rate)
  bl_opt = BlackLittermanOptimizer(returns_data, risk_free_rate=risk_free_rate)
  _ = MBTEOptimizer(returns_data, benchmark_data, risk_free_rate=risk_free_rate)

  n_assets = len(asset_names)
  P = np.zeros((2, n_assets)); P[0, 0] = 1; P[1, 1] = 1
  Q = np.array([0.02, -0.01]); omega = np.diag([0.001, 0.001])
  bl_opt.update_views(P, Q, omega)

  def calculate_portfolio_moments(w, ret_df):
    port = ret_df.dot(w)
    return float(skew(port)), float(kurtosis(port, fisher=True))

  def higher_moment_objective(w, exp_ret, cov, ret_df, risk_aversion=3.0, skew_penalty=0.1, kurt_penalty=0.05):
    covm = np.asarray(cov)
    r = float(np.dot(w, exp_ret))
    var = float(np.dot(w.T, np.dot(covm, w)))
    mv_util = r - 0.5 * risk_aversion * var
```

```
    port = ret_df.dot(w)
    ps = float(skew(port)); pk = float(kurtosis(port, fisher=True))
    skew_term = skew_penalty * min(0.0, ps) ** 2
    kurt_term = kurt_penalty * max(0.0, pk) ** 2
    return -(mv_util - skew_term - kurt_term)

hm_ports: Dict[str, np.ndarray] = {}

res_bl_hm = minimize(lambda w: higher_moment_objective(w, bl_opt.bl_returns, bl_opt.bl_cov, returns_data, bl_opt.risk_aversion),
                     x0=np.ones(n_assets) / n_assets, method="SLSQP",
                     bounds=tuple([(0, 1)] * n_assets), constraints=[{"type": "eq", "fun": lambda w: np.sum(w) - 1}])
hm_ports["BL_Higher_Moments"] = res_bl_hm.x if res_bl_hm.success else np.ones(n_assets) / n_assets

alpha = float(np.clip(blend_alpha, 0.0, 1.0))
gamma = max(float(te_gamma), 0.0)

def mbte_hm_obj(w):
    base_u = -higher_moment_objective(
        w,
        base_opt.mean_returns.values,
        base_opt.cov_matrix.values,
        returns_data,
        bl_opt.risk_aversion,
    )
    te_pen = tracking_error(w, returns_data, benchmark_data) ** 2
    score = alpha * base_u - (1.0 - alpha) * gamma * te_pen
    return -score

res_mbte_hm = minimize(mbte_hm_obj, x0=np.ones(n_assets) / n_assets, method="SLSQP",
                       bounds=tuple([(0, 1)] * n_assets), constraints=[{"type": "eq", "fun": lambda w: np.sum(w) - 1}])
hm_ports["MBTE_Higher_Moments"] = res_mbte_hm.x if res_mbte_hm.success else np.ones(n_assets) / n_assets

def comb_hm_obj(w):
    bl_u = -higher_moment_objective(
        w,
        bl_opt.bl_returns,
        bl_opt.bl_cov,
        returns_data,
        bl_opt.risk_aversion,
        0.15,
        0.08,
    )
    te_pen = tracking_error(w, returns_data, benchmark_data) ** 2
    score = alpha * bl_u - (1.0 - alpha) * gamma * te_pen
    return -score

res_comb_hm = minimize(comb_hm_obj, x0=np.ones(n_assets) / n_assets, method="SLSQP",
                       bounds=tuple([(0, 1)] * n_assets), constraints=[{"type": "eq", "fun": lambda w: np.sum(w) - 1}])
hm_ports["BL_MBTE_Higher_Moments"] = res_comb_hm.x if res_comb_hm.success else np.ones(n_assets) / n_assets

def mv_obj(w):
    covm = np.asarray(bl_opt.bl_cov)
    r = float(np.dot(w, bl_opt.bl_returns)); v = float(np.dot(w.T, np.dot(covm, w)))
    return -(r - 0.5 * bl_opt.risk_aversion * v)

res_mv = minimize(mv_obj, x0=np.ones(n_assets) / n_assets, method="SLSQP",
                  bounds=tuple([(0, 1)] * n_assets), constraints=[{"type": "eq", "fun": lambda w: np.sum(w) - 1}])
hm_ports["Traditional_MV"] = res_mv.x if res_mv.success else np.ones(n_assets) / n_assets

# Metrics
hm_results = pd.DataFrame(index=hm_ports.keys())
for name, w in hm_ports.items():
    if ("BL" in name and name != "Traditional_MV"):
        exp_ret, cov = bl_opt.bl_returns, bl_opt.bl_cov
    else:
        exp_ret, cov = base_opt.mean_returns.values, base_opt.cov_matrix.values
```

```
      ret, vol, shrp = portfolio_performance(w, exp_ret, cov, risk_free_rate)
      te = tracking_error(w, returns_data, benchmark_data)
      ps, pk = calculate_portfolio_moments(w, returns_data)
      hm_results.loc[name, ["Expected_Return", "Volatility", "Sharpe_Ratio", "Tracking_Error", "Skewness", "Excess_Kurtosis"]] = [ret *
100, vol * 100, shrp, te * 100, ps, pk]

  hm_alloc = pd.DataFrame({k: v for k, v in hm_ports.items()}, index=asset_names)

  charts = {
    "risk_return_bubble": {
      "points": [
        {"name": name,
         "x": float(hm_results.loc[name, "Volatility"]),
         "y": float(hm_results.loc[name, "Expected_Return"]),
         "size": float(abs(hm_results.loc[name, "Skewness"])) ,
         "color": float(hm_results.loc[name, "Excess_Kurtosis"]) }
        for name in hm_results.index.tolist()
      ],
      "x_label": "Volatility (%)",
      "y_label": "Expected Return (%)",
      "size_label": "|Skewness|",
      "color_label": "Excess Kurtosis"
    },
    "skew_bar": {
      "labels": hm_results.index.tolist(),
      "values": [float(v) for v in hm_results["Skewness"].tolist()]
    },
    "kurtosis_bar": {
      "labels": hm_results.index.tolist(),
      "values": [float(v) for v in hm_results["Excess_Kurtosis"].tolist()]
    },
    "allocations": {
      "assets": asset_names,
      "series": [
        {"name": name, "values": [float(v) for v in hm_ports[name].tolist()]}
        for name in hm_ports.keys()
      ]
    },
    "sharpe_vs_skew": {
      "points": [
        {"name": name,
         "x": float(hm_results.loc[name, "Skewness"]),
         "y": float(hm_results.loc[name, "Sharpe_Ratio"])}
        for name in hm_results.index.tolist()
      ],
      "x_label": "Skewness",
      "y_label": "Sharpe Ratio"
    }
  }

  return {
    "results": df_to_split_json(hm_results.round(4)),
    "allocations": df_to_split_json((hm_alloc * 100).round(2)),
    "charts": charts,
  }


# ----------------------------
# Hybrid helpers + solver
# ----------------------------

def _benchmark_weights(
  bench_df: pd.DataFrame,
  mode: str = "anti_corr",
  override: Optional[np.ndarray] = None,
) -> np.ndarray:
```

```
    """
    Return non-negative weights φ over benchmarks that sum to 1.
    - 'uniform'     : equal weights
    - 'anti_corr'   : weight a benchmark inversely to its average correlation to others
    """
    K = bench_df.shape[1]
    if K == 0:
        return np.array([])
    if override is not None:
        w = np.asarray(override, dtype=float).reshape(-1)
        if w.shape[0] == K and np.all(w >= 0):
            total = w.sum()
            if total > 0:
                return w / total
    # fallback to mode-based weights when no valid override is provided
    if mode == "uniform" or K == 1:
        return np.ones(K) / K
    C = bench_df.corr().values
    avg_abs = np.mean(np.abs(C - np.eye(K)), axis=1)
    inv = 1.0 / np.clip(avg_abs, 1e-6, None)
    w = inv / inv.sum()
    return w


def _factor_exposure_matrix(
    asset_ret: pd.DataFrame,
    factor_ret: pd.DataFrame,
    ridge_lambda: float = 0.0,
) -> np.ndarray:
    """
    OLS loadings of assets on factors (B in R_t^assets ≈ B F_t).
    Returns matrix B of shape (n_assets, n_factors).
    """
    X = factor_ret.dropna()
    Y = asset_ret.loc[X.index].dropna(axis=1, how="any")
    common = X.index.intersection(Y.index)
    X = X.loc[common]; Y = Y.loc[common]
    if X.empty or Y.empty:
        return np.zeros((asset_ret.shape[1], factor_ret.shape[1]))
    X_ = np.column_stack([np.ones(len(X)), X.values])
    XtX = X_.T @ X_
    if ridge_lambda > 0:
        ridge = np.eye(XtX.shape[0]) * float(ridge_lambda)
        ridge[0, 0] = 0.0  # keep intercept unbiased
        XtX = XtX + ridge
    XtY = X_.T @ Y.values
    try:
        B_full = np.linalg.solve(XtX, XtY)
    except np.linalg.LinAlgError:
        B_full = np.linalg.pinv(XtX) @ XtY
    B = B_full[1:, :].T  # drop intercept -> (N, F)
    out = np.zeros((asset_ret.shape[1], factor_ret.shape[1]))
    asset_idx = {a:i for i,a in enumerate(Y.columns)}
    for j,a in enumerate(asset_ret.columns):
        if a in asset_idx:
            out[j,:] = B[asset_idx[a], :]
    return out


class HybridBLMBTE:
    """
    maximize:  U(w) = w' μ_BL - 0.5 λ w' Σ_BL w
                      - γ * sum_k φ_k * TE_k(w)^2
                      - δ * ‖ (B' w - f_target) ‖^2
               s.t.  1'w = 1, 0 <= w <= 1,  TE_k(w) <= τ_k (optional)
```

```
  where TE_k(w) = std( r_p - r_bk ) * sqrt(252)
  """
  def __init__(
      self,
      asset_ret: pd.DataFrame,
      bench_ret: pd.DataFrame,
      bl_mu: np.ndarray,
      bl_cov: np.ndarray,
      risk_aversion: float = 3.0,
      gamma: float = 1.0,
      bench_weight_mode: str = "anti_corr",
      alpha: float = 0.6,
      te_caps: Optional[Dict[str, float]] = None,
      bench_weights: Optional[List[float]] = None,
      factor_B: Optional[np.ndarray] = None,
      factor_target: Optional[np.ndarray] = None,
      delta: float = 0.0,
      risk_free_rate: float = 0.02,
  ):
      self.R = asset_ret
      self.BR = bench_ret
      self.mu = np.asarray(bl_mu).reshape(-1)
      self.S = np.asarray(bl_cov)
      self.lmbd = float(risk_aversion)
      self.gamma = float(gamma)
      self.delta = float(delta)
      self.rfr = float(risk_free_rate)
      self.alpha = float(np.clip(alpha, 0.0, 1.0))

      self.assets = list(asset_ret.columns)
      self.benches = list(bench_ret.columns)
      self.N = len(self.assets)
      self.K = len(self.benches)

      override_weights = None
      if bench_weights is not None:
          override_weights = np.asarray(bench_weights, dtype=float)
      self.phi = _benchmark_weights(bench_ret, bench_weight_mode, override=override_weights) if self.K > 0 else np.array([])
      self.te_caps = te_caps or {}

      self.B = factor_B
      self.f_target = None if factor_B is None else (np.zeros(factor_B.shape[1]) if factor_target is None else np.asarray(factor_target).reshape(-
1))

  def _te_vec(self, w: np.ndarray) -> np.ndarray:
      port = self.R.values @ w
      TEs = []
      for k in range(self.K):
          te = np.std(port - self.BR.values[:, k]) * np.sqrt(252.0)
          TEs.append(float(te))
      return np.array(TEs) if TEs else np.array([])

  def _objective(self, w: np.ndarray) -> float:
      # BL mean-variance utility
      ret = float(w @ self.mu)
      var = float(w @ (self.S @ w))
      bl_util = ret - 0.5 * self.lmbd * var

      # multi-benchmark TE penalty
      te_pen = 0.0
      if self.K > 0 and self.gamma > 0:
          te_k = self._te_vec(w)
          te_pen = float(self.gamma * np.sum(self.phi * (te_k ** 2)))

      # factor neutrality/target penalty
```

```
        fact_pen = 0.0
        if self.B is not None and self.delta > 0.0:
            exposure = self.B.T @ w  # (F,)
            diff = exposure - self.f_target
            fact_pen = float(self.delta * (diff @ diff))
        score = self.alpha * bl_util - (1.0 - self.alpha) * te_pen - fact_pen
        return -score

    def _constraints(self):
        cons = [{"type": "eq", "fun": lambda w: np.sum(w) - 1.0}]
        # TE caps per benchmark (inequality: cap - TE_k(w) >= 0)
        if self.K > 0 and len(self.te_caps) > 0:
            name_to_idx = {n:i for i,n in enumerate(self.benches)}
            for name, cap in self.te_caps.items():
                if name in name_to_idx:
                    idx = name_to_idx[name]
                    cons.append({
                        "type": "ineq",
                        "fun": (lambda w, i=idx, c=float(cap): c - self._te_vec(w)[i])
                    })
        return cons

    def solve(self, x0: Optional[np.ndarray] = None, bounds: Optional[List[tuple]] = None) -> np.ndarray:
        if x0 is None:
            x0 = np.ones(self.N) / self.N
        if bounds is None:
            bounds = [(0.0, 1.0)] * self.N

        res = minimize(self._objective, x0=x0, method="SLSQP",
                       bounds=bounds, constraints=self._constraints(),
                       options={"maxiter": 1000})
        if not res.success:
            w = np.clip(res.x if res.x is not None else x0, 0, 1)
            w = w / w.sum()
            return w
        w = np.clip(res.x, 0, 1); w = w / w.sum()
        return w


def hybrid_analysis(
    returns_data: pd.DataFrame,
    benchmark_data: pd.DataFrame,
    sector_factors: Optional[pd.DataFrame],
    style_factors: Optional[pd.DataFrame],
    risk_free_rate: float,
    bl_risk_aversion: float = 3.0,
    bl_tau: float = 0.05,
    bench_weight_mode: str = "anti_corr",
    gamma: float = 1.0,
    te_caps: Optional[Dict[str, float]] = None,
    delta: float = 0.0,
    factor_target: Optional[List[float]] = None,
):
    """
    Builds BL μ, Σ; estimates factor loadings; solves HybridBLMBTE.
    """
    # --- BL prior (reuse BL class for μ_BL and Σ_BL) ---
    bl = BlackLittermanOptimizer(returns_data, risk_aversion=bl_risk_aversion, tau=bl_tau, risk_free_rate=risk_free_rate)
    n_assets = returns_data.shape[1]
    if n_assets >= 2:
        P = np.zeros((2, n_assets)); P[0,0]=1; P[1,1]=1
        Q = np.array([0.02, -0.01]); omega = np.diag([0.001, 0.001])
        bl.update_views(P, Q, omega)
    else:
        bl.bl_returns = bl.equilibrium_returns.copy()
        bl.bl_cov = bl.cov_matrix.values.copy()
```

```
# --- Factor matrix B using sector + style ---
factor_B = None
F = None
if (sector_factors is not None and not sector_factors.empty) or (style_factors is not None and not style_factors.empty):
    F = pd.concat([sector_factors, style_factors], axis=1).dropna()
    if not F.empty:
        factor_B = _factor_exposure_matrix(returns_data, F)

f_target_arr = None
if factor_B is not None and factor_target is not None:
    f_target_arr = np.asarray(factor_target).reshape(-1)
    if f_target_arr.shape[0] != factor_B.shape[1]:
        # size mismatch; ignore target gracefully
        f_target_arr = None

# --- Solve hybrid ---
hybr = HybridBLMBTE(
    asset_ret=returns_data,
    bench_ret=benchmark_data,
    bl_mu=bl.bl_returns,
    bl_cov=bl.bl_cov,
    risk_aversion=bl_risk_aversion,
    gamma=gamma,
    bench_weight_mode=bench_weight_mode,
    te_caps=te_caps,
    factor_B=factor_B,
    factor_target=f_target_arr,
    delta=delta,
    risk_free_rate=risk_free_rate,
)
w = hybr.solve()

# metrics
port_ret, port_vol, port_sr = portfolio_performance(w, bl.bl_returns, bl.bl_cov, risk_free_rate)
te_k = []
for col in benchmark_data.columns:
    te_k.append(float((returns_data.dot(w) - benchmark_data[col]).std()*np.sqrt(252)*100))
te_avg = float(np.mean(te_k)) if len(te_k)>0 else float('nan')

results = pd.DataFrame(
    {
        "Expected_Return":[port_ret*100],
        "Volatility":[port_vol*100],
        "Sharpe_Ratio":[port_sr],
        "Tracking_Error_Avg":[te_avg],
    },
    index=["BL_MBTE_Hybrid"]
)
alloc = pd.Series(w, index=returns_data.columns, name="BL_MBTE_Hybrid")

charts = {
    "risk_return_point": {
        "points": [{"name":"BL_MBTE_Hybrid","x":float(results["Volatility"].iloc[0]),"y":float(results["Expected_Return"].iloc[0])}],
        "x_label":"Volatility (%)","y_label":"Expected Return (%)"
    },
    "te_per_benchmark": {
        "labels": list(benchmark_data.columns),
        "values": te_k
    },
    "allocations": {
        "assets": alloc.index.tolist(),
        "series": [{"name":"BL_MBTE_Hybrid","values":[float(x) for x in alloc.values.tolist()]}]
    }
}
```

```
    return {
        "results": df_to_split_json(results.round(4)),
        "allocations": df_to_split_json((alloc.to_frame()*100).round(2)),
        "charts": charts,
        "hyperparams": {
            "risk_aversion": bl_risk_aversion,
            "gamma": gamma,
            "bench_weight_mode": bench_weight_mode,
            "delta_factor_penalty": delta,
            "te_caps": te_caps or {},
            "factors_used": list(F.columns) if F is not None else [],
        }
    }


# ------------------------------
# Master runner
# ------------------------------

def run_complete_analysis(
    sp500_tickers: List[str],
    sector_factors: List[str],
    style_factors: List[str],
    benchmarks: Union[Dict[str, str], List[str]],
    start: str = "2015-01-01",
    end: Optional[str] = None,
    which: Optional[List[str]] = None,
    hybrid_cfg: Optional["HybridConfig"] = None,
):
    if which is None:
        which = ["mean_variance", "max_sharpe", "higher_moments"]

    assets_ret, sector_ret, style_ret, bench_ret, rfr_annual = load_market_data(
        sp500_tickers, sector_factors, style_factors, benchmarks, start, end
    )

    if assets_ret.empty or bench_ret.empty:
        raise HTTPException(status_code=422, detail="No data returned; verify tickers and date range.")

    out: Dict[str, Any] = {
        "risk_free_annual": rfr_annual,
        "meta": {
            "n_assets": int(assets_ret.shape[1]),
            "n_days": int(assets_ret.shape[0]),
            "date_range": [str(assets_ret.index.min().date()), str(assets_ret.index.max().date())],
            "benchmarks": list(bench_ret.columns),
            "assets": list(assets_ret.columns),
        },
    }

    if "mean_variance" in which:
        out["mean_variance"] = mean_variance_analysis(assets_ret, bench_ret, rfr_annual)

    if "max_sharpe" in which:
        out["max_sharpe"] = max_sharpe_analysis(assets_ret, bench_ret, rfr_annual)

    if "higher_moments" in which:
        out["higher_moments"] = higher_moments_analysis(assets_ret, bench_ret, rfr_annual)

    if (hybrid_cfg is not None) or ("hybrid" in which):
        hc = hybrid_cfg or HybridConfig()
        out["hybrid"] = hybrid_analysis(
            returns_data=assets_ret,
            benchmark_data=bench_ret,
            sector_factors=sector_ret,
            style_factors=style_ret,
```

```
            risk_free_rate=rfr_annual,
            bl_risk_aversion=hc.risk_aversion,
            bl_tau=hc.tau,
            bench_weight_mode=hc.bench_weight_mode,
            gamma=hc.gamma,
            te_caps=hc.te_caps,
            delta=hc.delta_factor_penalty,
            factor_target=hc.factor_target,
        )

    return out


# ----------------------------
# FastAPI models & app
# ----------------------------

class HybridConfig(BaseModel):
    bench_weight_mode: Optional[str] = Field("anti_corr", description="one of {'uniform','anti_corr'}")
    gamma: Optional[float] = Field(1.0, description="penalty weight for TE^2")
    risk_aversion: Optional[float] = Field(3.0, description="BL risk aversion λ")
    tau: Optional[float] = Field(0.05, description="BL τ (scales prior covariance)")
    te_caps: Optional[Dict[str, float]] = Field(None, description="per-benchmark TE caps (e.g., {'SPY':0.08})")
    delta_factor_penalty: Optional[float] = Field(0.0, description="penalty weight on factor exposure deviation")
    factor_target: Optional[List[float]] = Field(None, description="target factor exposures (len = #sector + #style)")
    blend_alpha: Optional[float] = Field(0.6, description="weight on BL utility vs TE penalty (0-1)")
    bench_weights: Optional[List[float]] = Field(None, description="optional explicit benchmark weights φ")
    factor_reg: Optional[float] = Field(0.0, description="ridge regularization applied to factor regressions")


class AnalyzeRequest(BaseModel):
    sp500_tickers: List[str] = Field(..., description="List of asset tickers (e.g., S&P 500 subset)")
    sector_factors: List[str] = Field(..., description="Sector ETF tickers")
    style_factors: List[str] = Field(..., description="Style ETF tickers")
    benchmarks: Union[Dict[str, str], List[str]] = Field(..., description="Benchmark tickers or mapping to descriptions")
    start: Optional[str] = Field("2015-01-01", description="Start date YYYY-MM-DD")
    end: Optional[str] = Field(None, description="End date YYYY-MM-DD (or null for today)")
    which: Optional[List[str]] = Field(default_factory=lambda: ["mean_variance", "max_sharpe", "higher_moments"], description="Analyses
to run")
    hybrid: Optional[HybridConfig] = Field(None, description="Optional hybrid BL+MBTE configuration")


app = FastAPI(title="BL + MBTE Portfolio API", version="1.1.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],      # includes POST, OPTIONS
    allow_headers=["*"],      # includes Content-Type, Authorization, etc.
)

@app.get("/health")
def health():
    return {"status": "ok"}


@app.post("/analyze")
def analyze(req: AnalyzeRequest):
    try:
        result = run_complete_analysis(
            sp500_tickers=req.sp500_tickers,
            sector_factors=req.sector_factors,
            style_factors=req.style_factors,
            benchmarks=req.benchmarks,
            start=req.start or "2015-01-01",
```

```
            end=req.end,
            which=req.which,
            hybrid_cfg=req.hybrid,
        )
        return JSONResponse(content=result)
    except HTTPException as he:
        raise he
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Analysis failed: {e}")


if __name__ == "__main__":
    import uvicorn
    uvicorn.run("api_improv:app", host="0.0.0.0", port=8100, reload=True)
```